Variational Auto Encoders

Introduction to Deep Learning

Fall 2026

Attendance: @

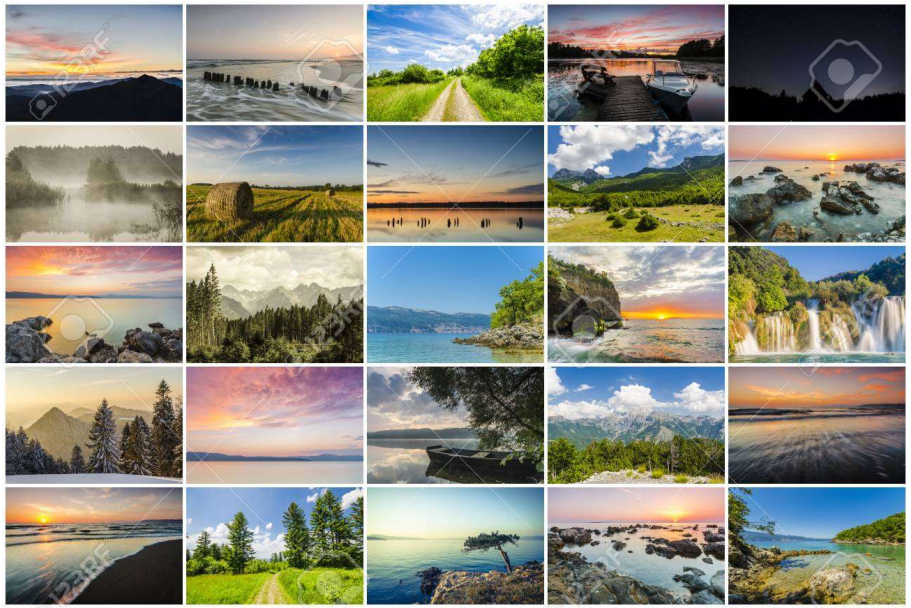
A new problem





- From a large collection of images of faces, can a network learn to *generate* new portraits
  - Generate samples from the distribution of “face” images
    - How do we even characterize this distribution?

A new problem

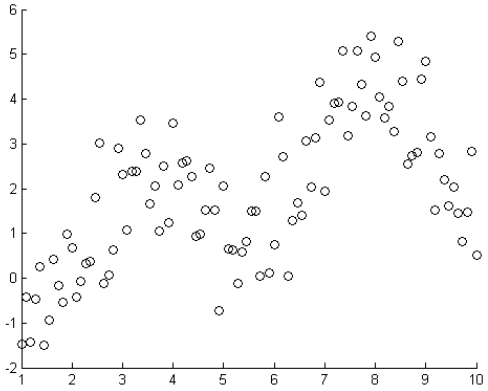


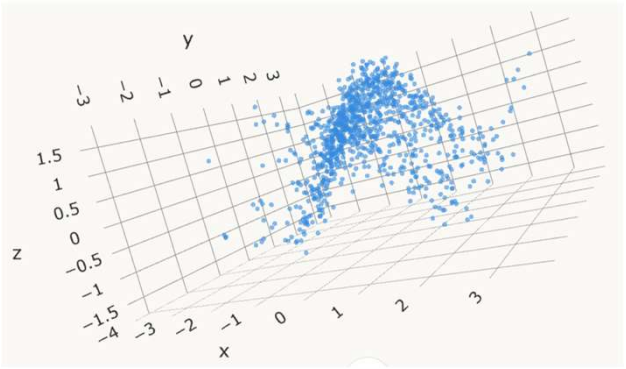


- From a large collection of landscapes, can a network learn to *generate* new landscape pictures
  - Generate samples from the distribution of “landscape” images
    - How do we even characterize this distribution?

The distribution of data

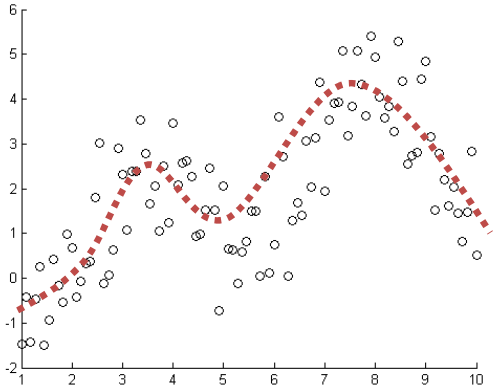
- The distribution of any *structured* data, occupies only a restricted region of the overall space
  - E.g. face images, or landscapes or any other category of data
  - This region may have any shape
- **Why:** Structure implies reduction in degrees of freedom
  - Reduction in degrees of freedom implies restriction of possibilities
  - Only noise occupies the entire space
- **Challenge of generation**: Characterizing this distribution and sampling from it

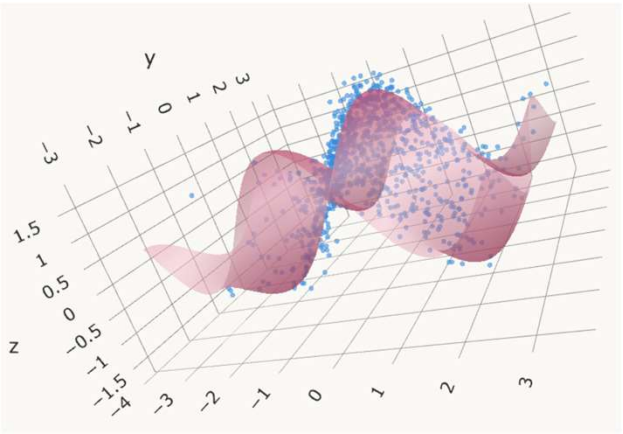




Modeling the distribution of data

- **Hypothesis:** The data are distributed about a non-linear low-dimensional manifold (surface) in the high dimensional data space
  - The manifold characterizes the restricted region in which the target data reside
  - The principal components of all instances of the target class of data lie on this manifold
- To generate data for this class, we must select a point on this manifold

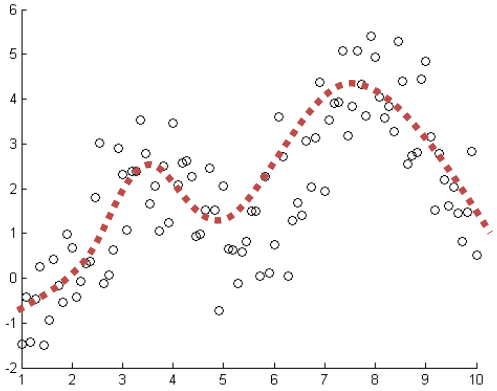


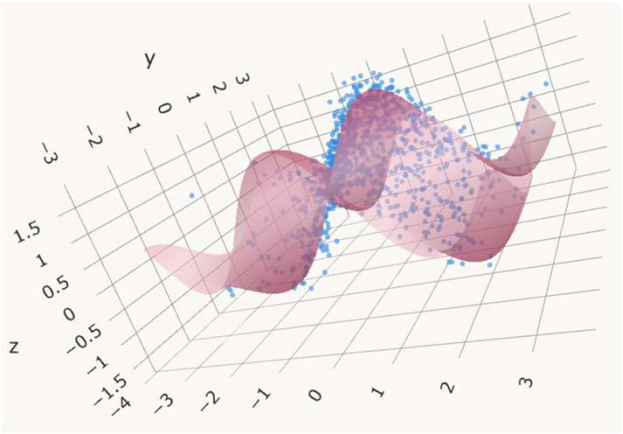


- **Problems:**
  - Characterizing the manifold
  - Having a good strategy for selecting points from it

Modeling the distribution of data

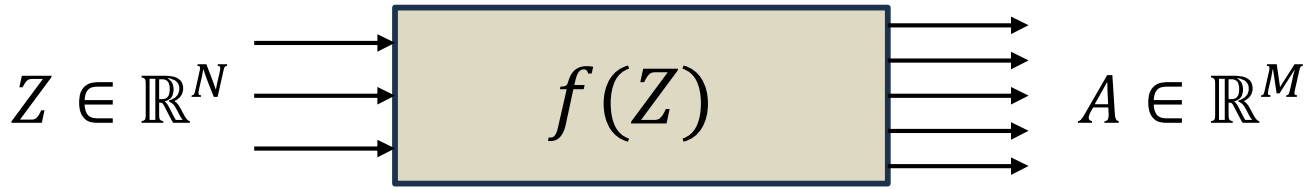
- **Hypothesis:** The data are distributed about a non-linear low-dimensional manifold (surface) in the high dimensional data space
  - The manifold characterizes the restricted region in which the target data reside
  - The principal components of all instances of the target class of data lie on this manifold
- To generate data for this class, we must select a point on this manifold



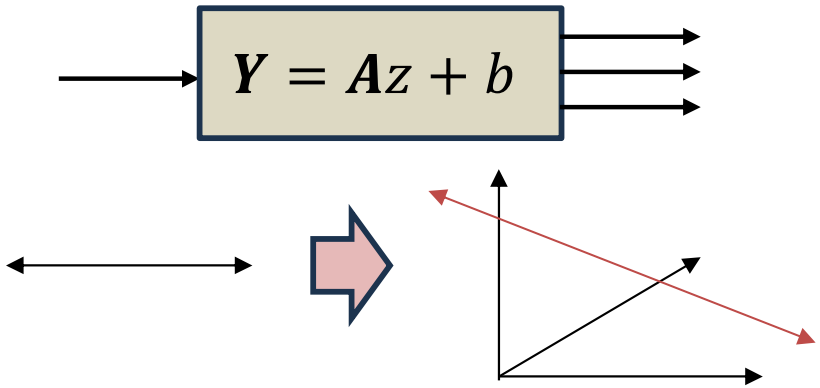


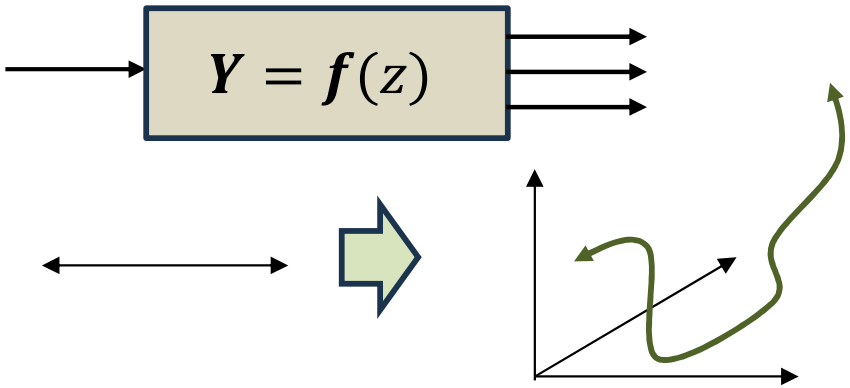
- **Problems:**
  - Characterizing the manifold
  - Having a good strategy for selecting points from it

Recap: A general property of dimensionality-increasing functions



- A function with N inputs and M outputs, M > N simply transforms the N-dimensional input space into an N-dimensional surface in M-dimensional space

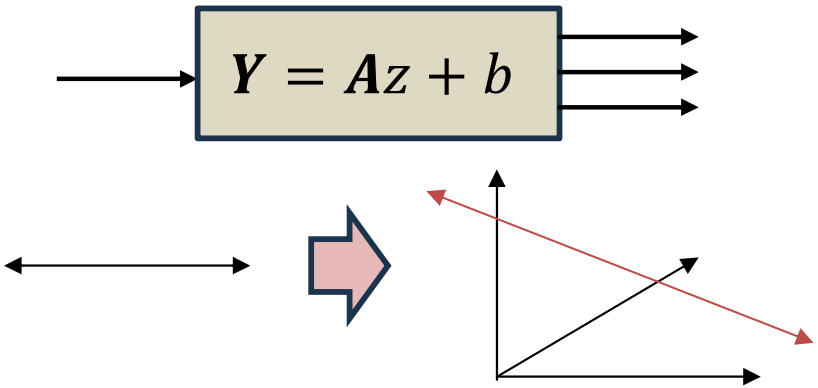


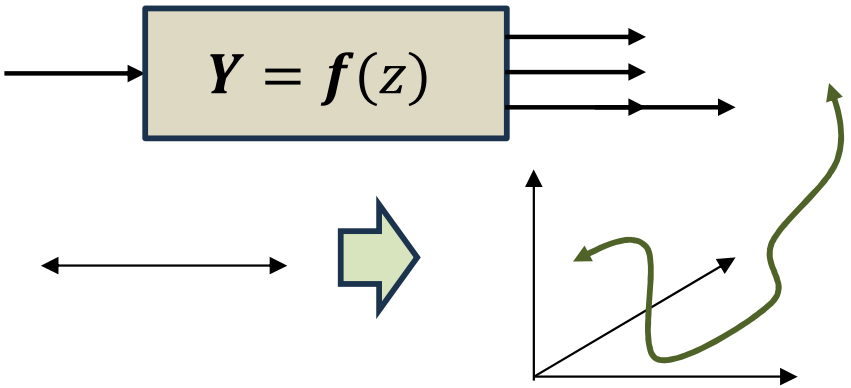


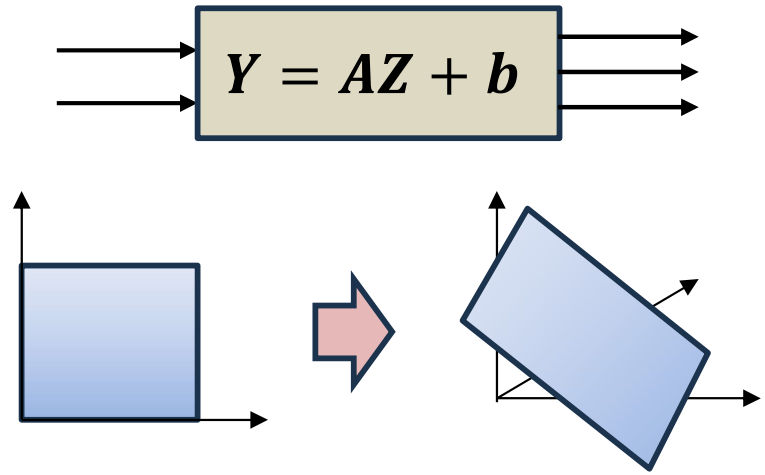
- A **linear** or **affine** function converts the N-dimensional input space into an N-dimensional hyperplane in M-dimensional space

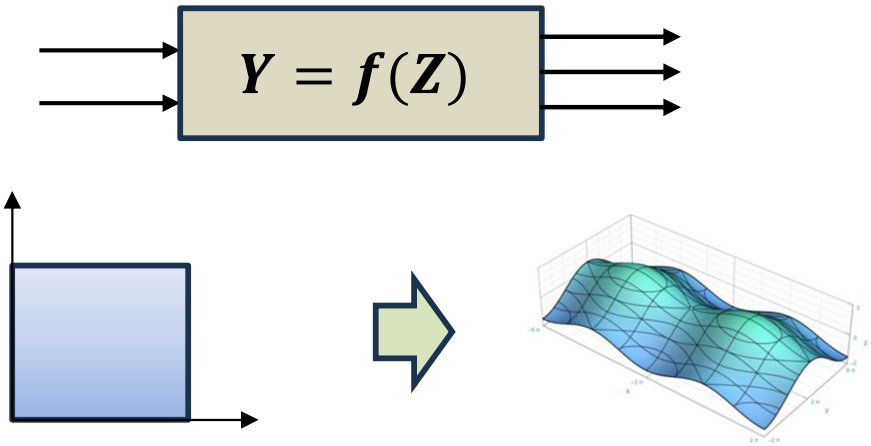
- A ***non*-linear** function converts the N-dimensional input space into an N-dimensional curved surface (manifold) in M-dimensional space

Dimensionality-increasing functions







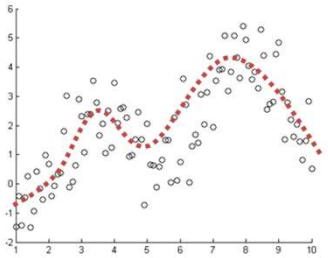


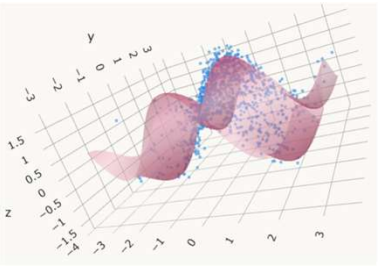
- A **linear** or **affine** function converts the N-dimensional input space into an N-dimensional hyperplane in M-dimensional space

- A ***non*-linear** function converts the N-dimensional input space into an N-dimensional curved surface (manifold) in M-dimensional space

Modelling the distribution of data

- **Distribution:** A fuzzy scatter centered on a low dimensional non-linear surface manifold

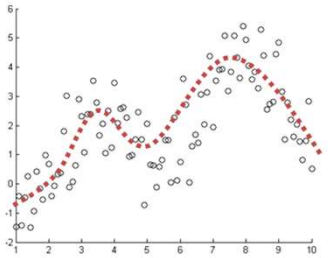


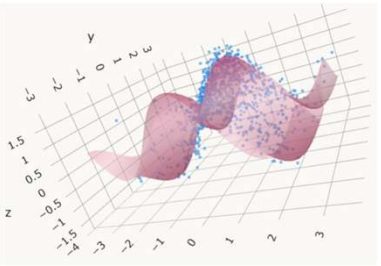


- **Approach:**
  - First model the distribution on the manifold itself
  - Then add noise for the off-manifold scatter

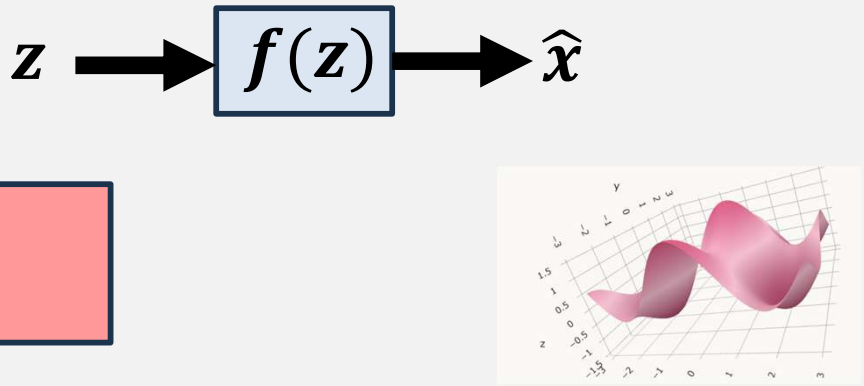
Modelling the distribution of data

- **Distribution:** A fuzzy scatter centered on a low non-linear surface manifold





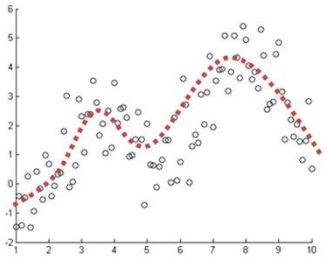
- **Approach:**
  - **First model the distribution on the manifold itself**
  - Then add noise for the off-manifold scatter

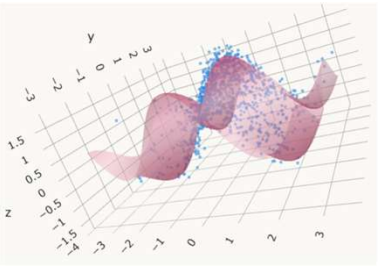


- **The Manifold:** Transform an N-dimensional "latent" space into the N-dimensional manifold in the M-dimensional data space using a learned non-linear function
  - The data distribution on the N-dimensional latent space transforms into a different distribution on the N-dimensional manifold

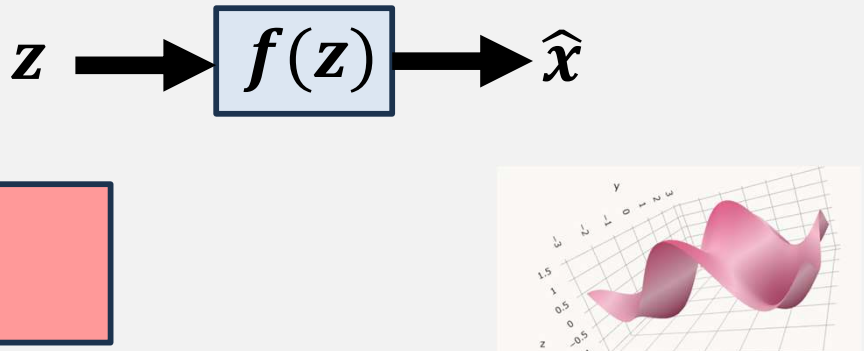
Modelling the distribution of data

- **Distribution:** A fuzzy scatter centered on a low non-linear surface manifold

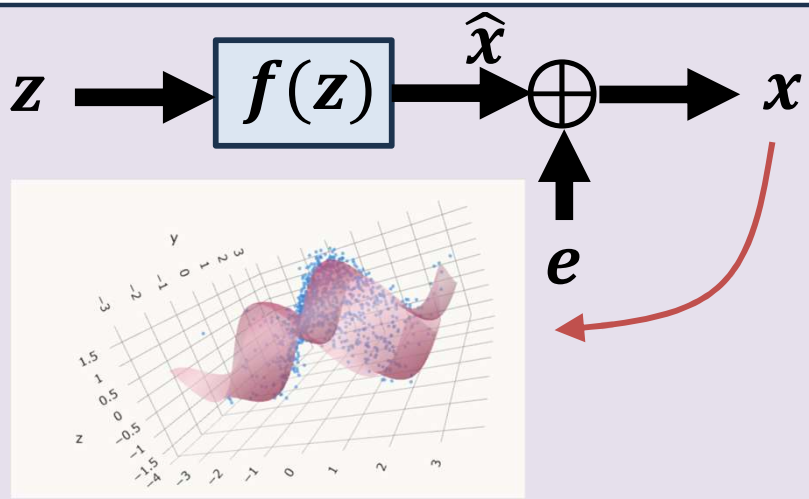




- **Approach:**
  - First model the distribution on the manifold itself
  - Then add noise for the off-manifold scatter

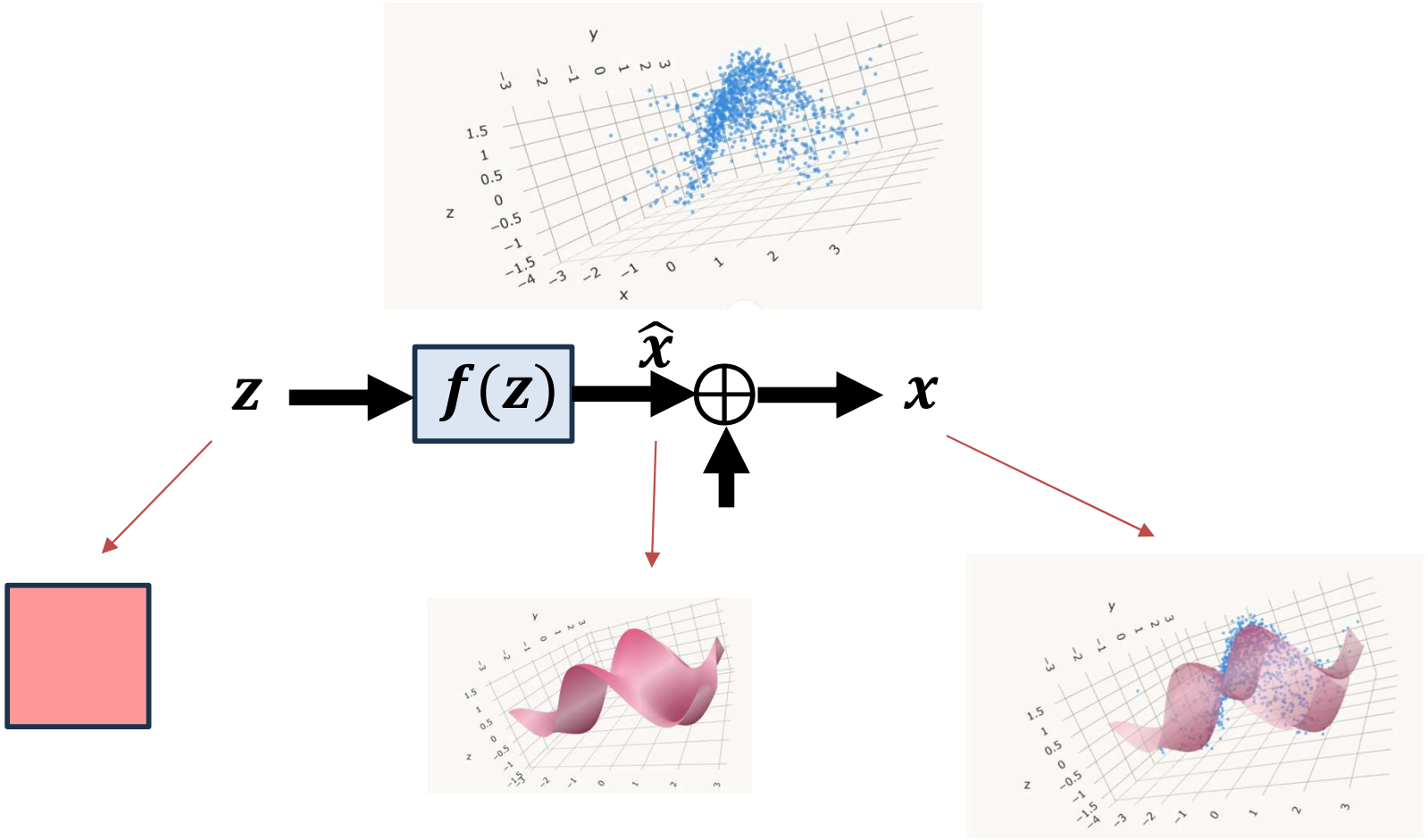


- **The Manifold:** Transform an N-dimensional "latent" space into the N-dimensional manifold in the M-dimensional data space using a learned non-linear function
  - The data distribution on the N-dimensional latent space transforms into a different distribution on the N-dimensional manifold



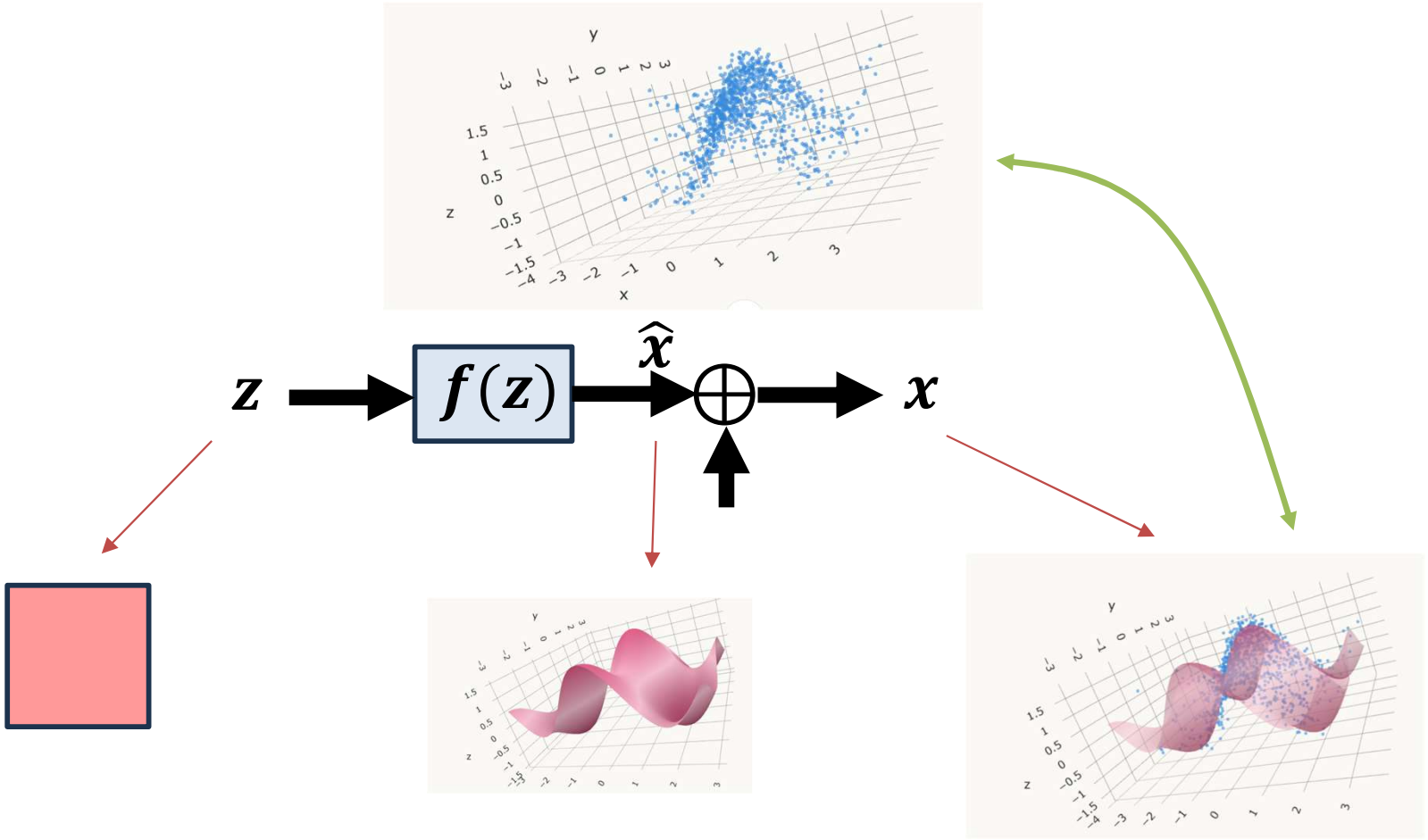
- **The "fuzz":** Add noise to account for the off-manifold "fuzz"
  - If the manifold captures all the structure in the data, this fuzz will be Gaussian

Modelling the distribution of data



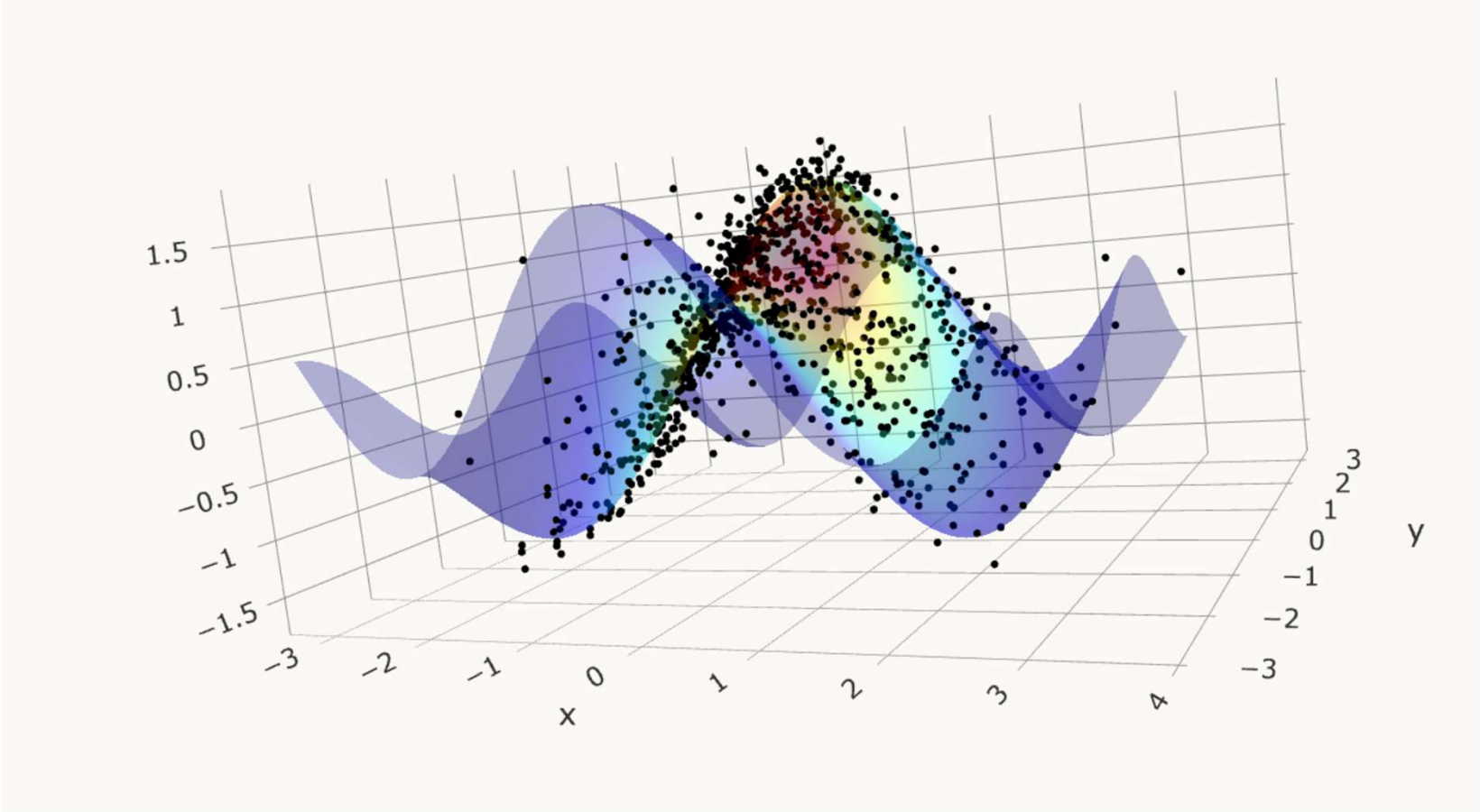
- Something is missing…

Modelling the distribution of data



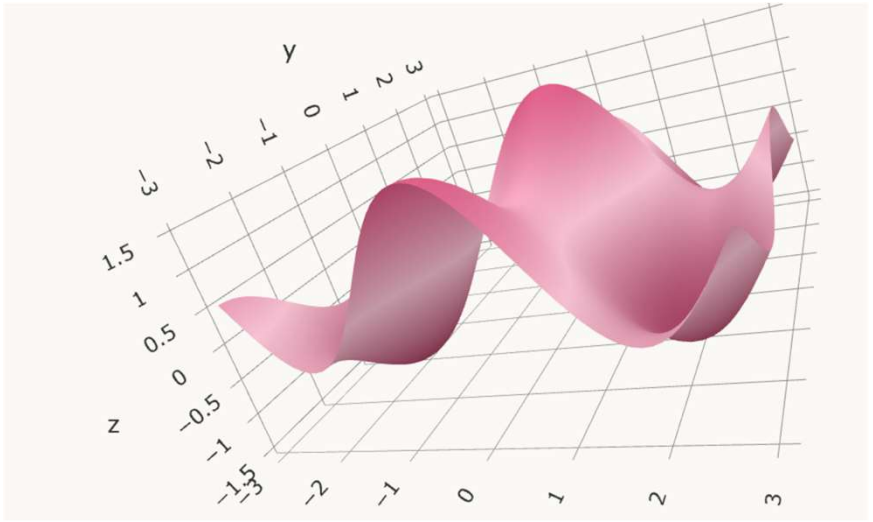
- Something is missing…
  - The actual *distribution* of the data is not represented yet…

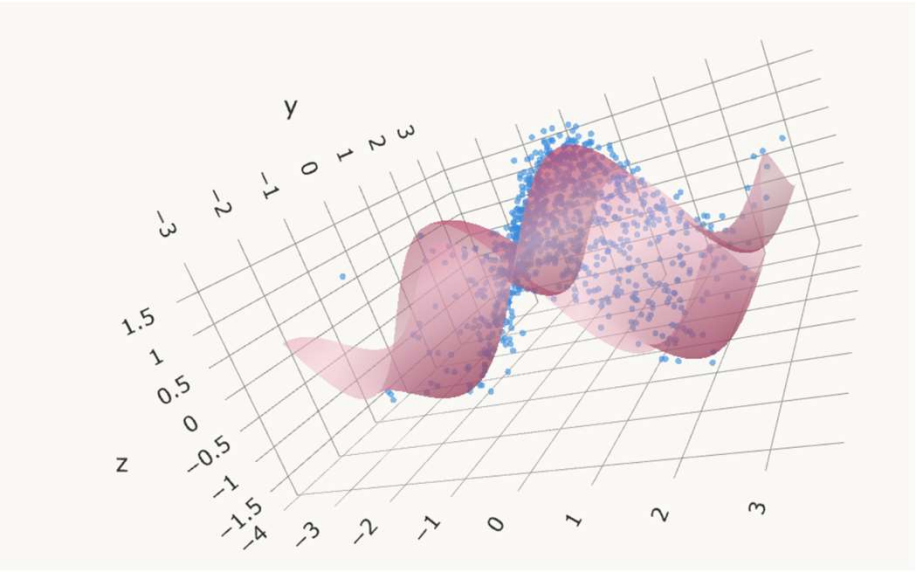
The actual distribution of the data

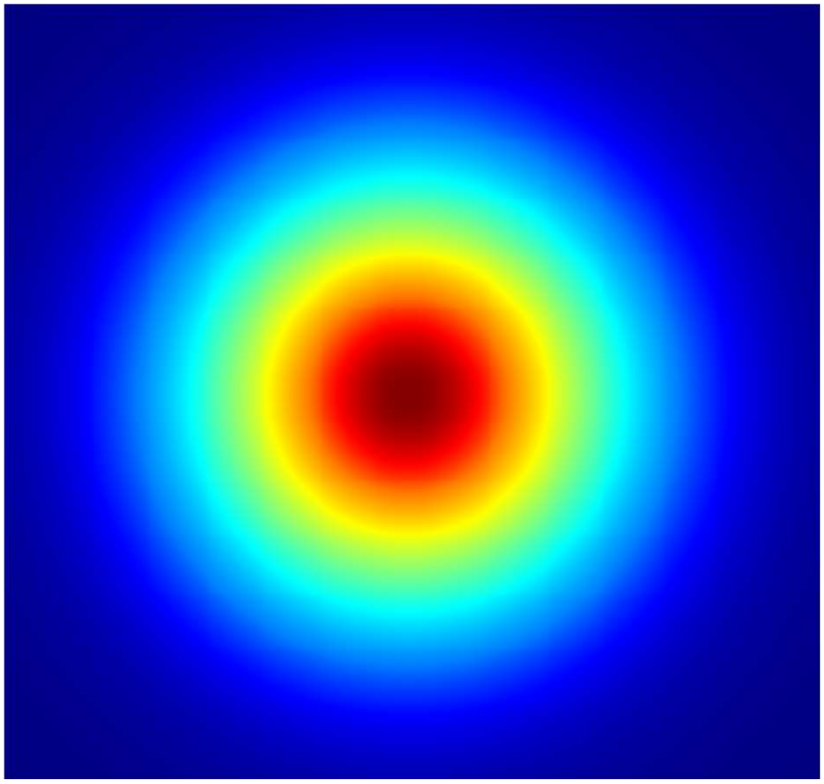


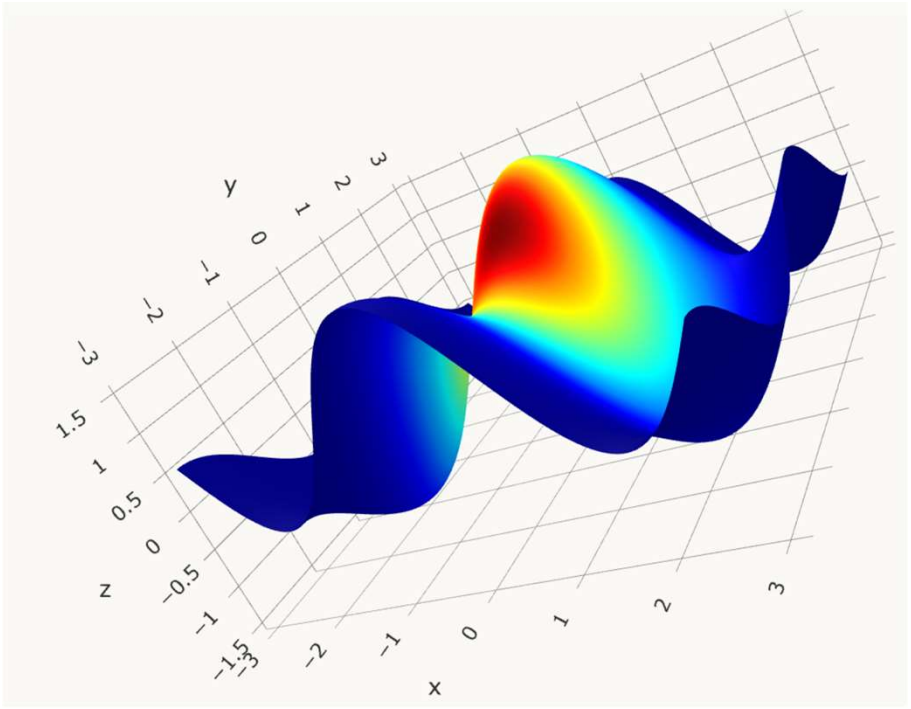
- d

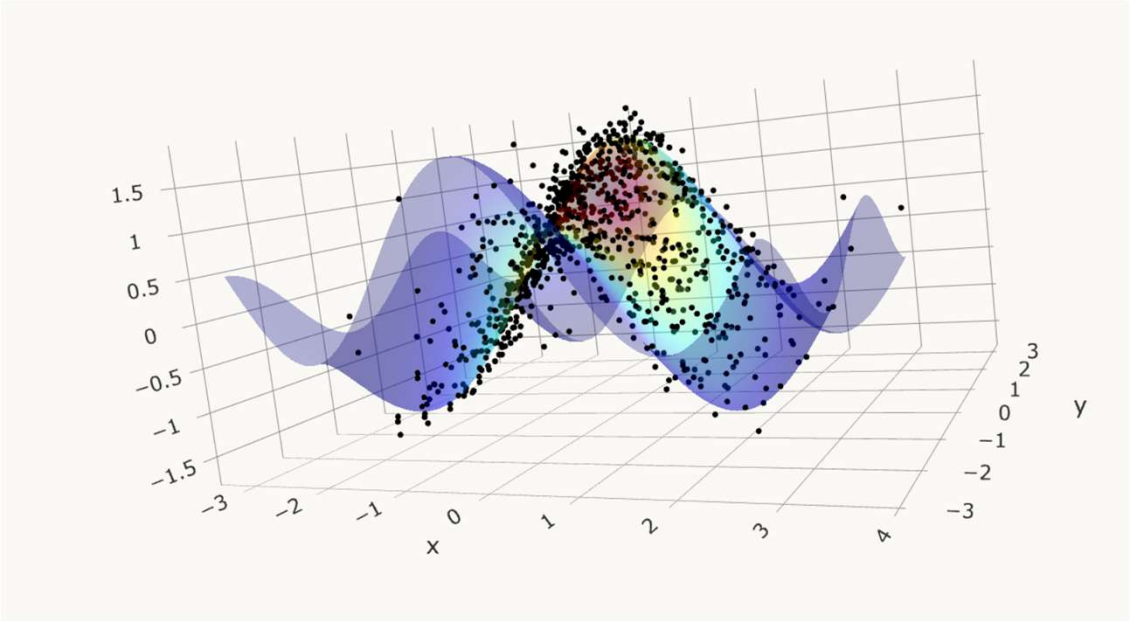

y
x
z


y
z

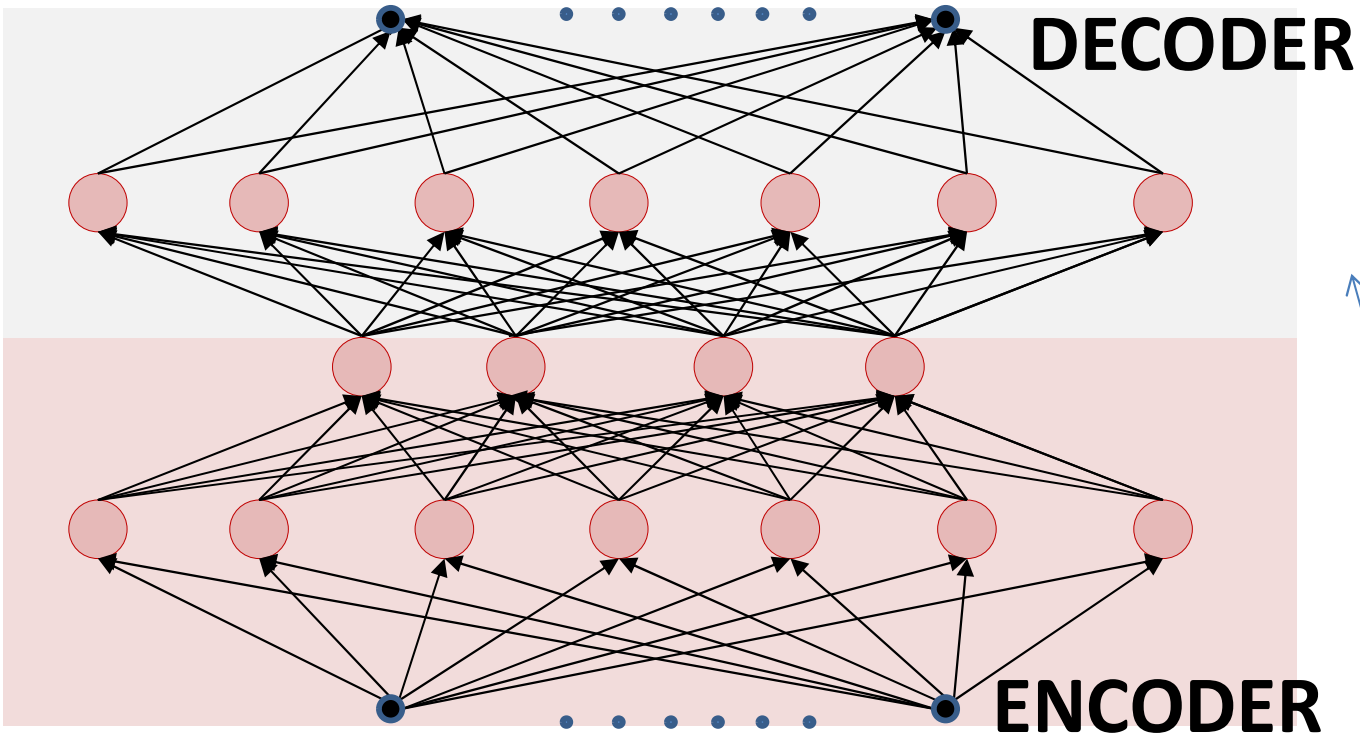

y
z

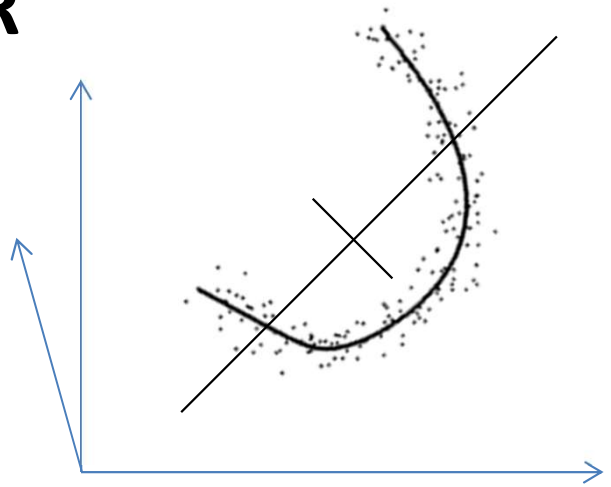



y
3
2
1
0
−1
−2
−3
1.5
1
0.5
0
−0.5
−1
−1.5
z
−3
−2
−1
0
1
2
3
x


1.5
1
0.5
0
−0.5
−1
−1.5
−3
−2
−1
0
1
2
3
4
x
3
2
1
0
−1
−2
−3
y

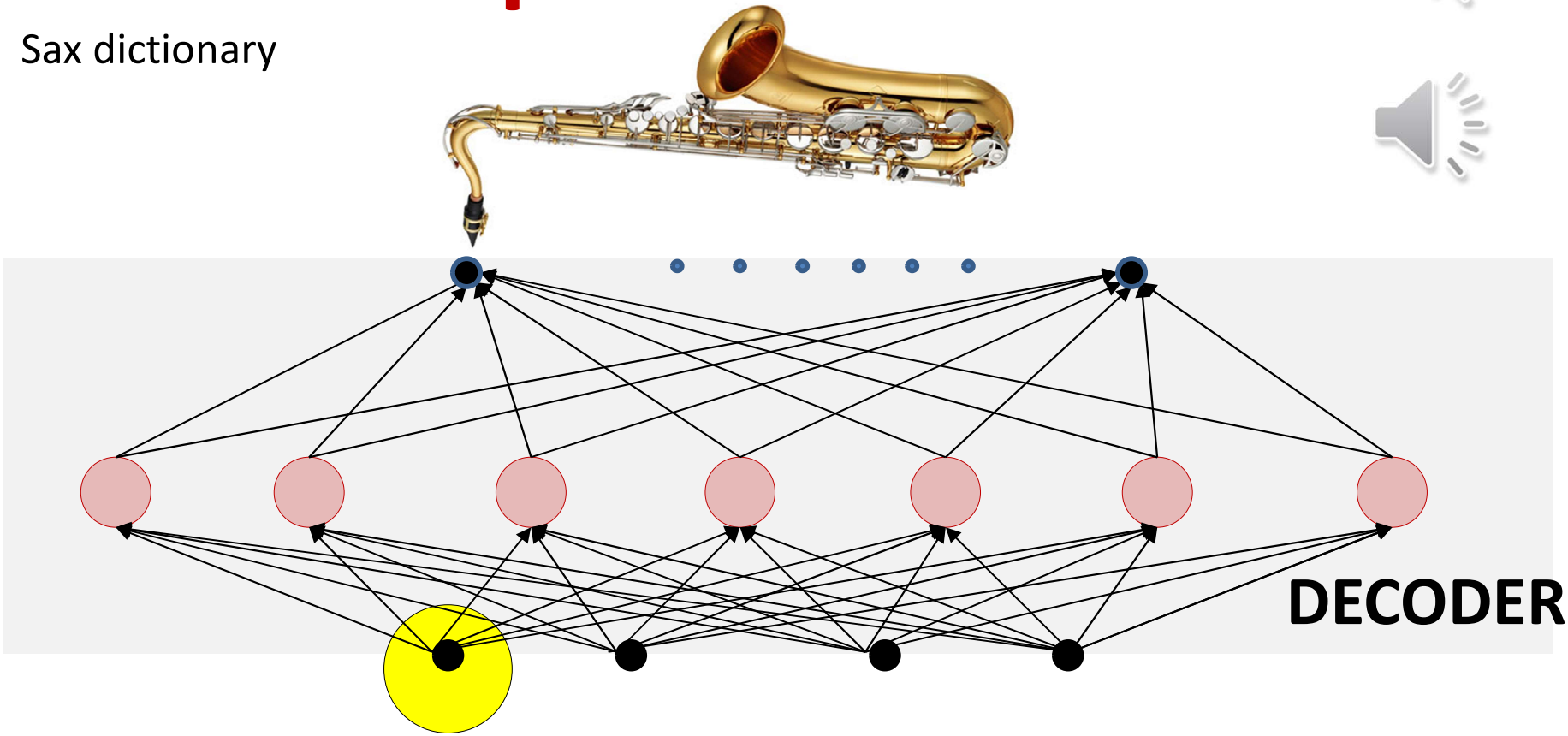
Recall : The AE





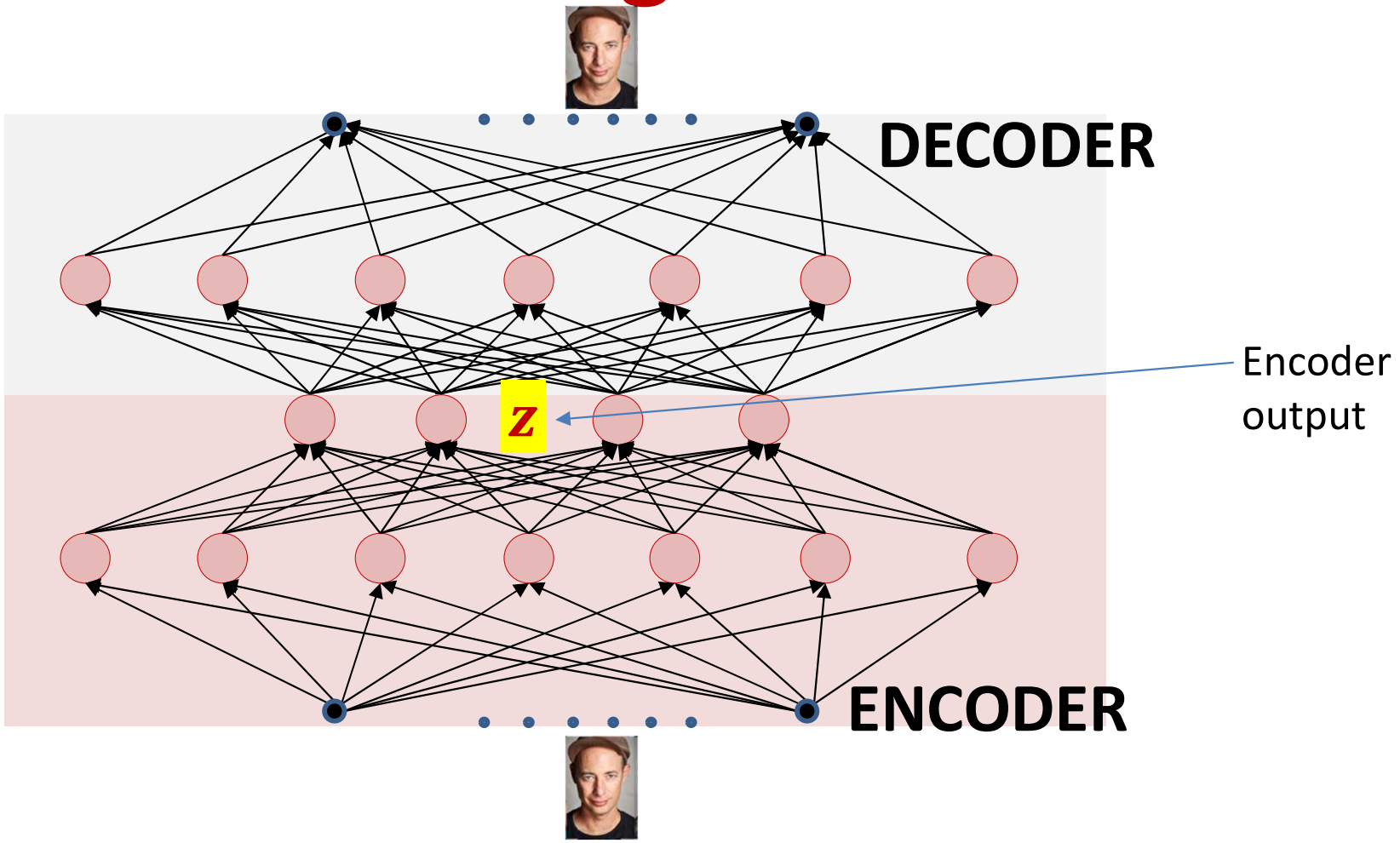
- The (decoder of the) autoencoder captures the underlying manifold of the data
  - "Non linear" PCA
  - Deeper networks can capture more complicated manifolds
    - "Deep" autoencoders

Recap : The Decoder:



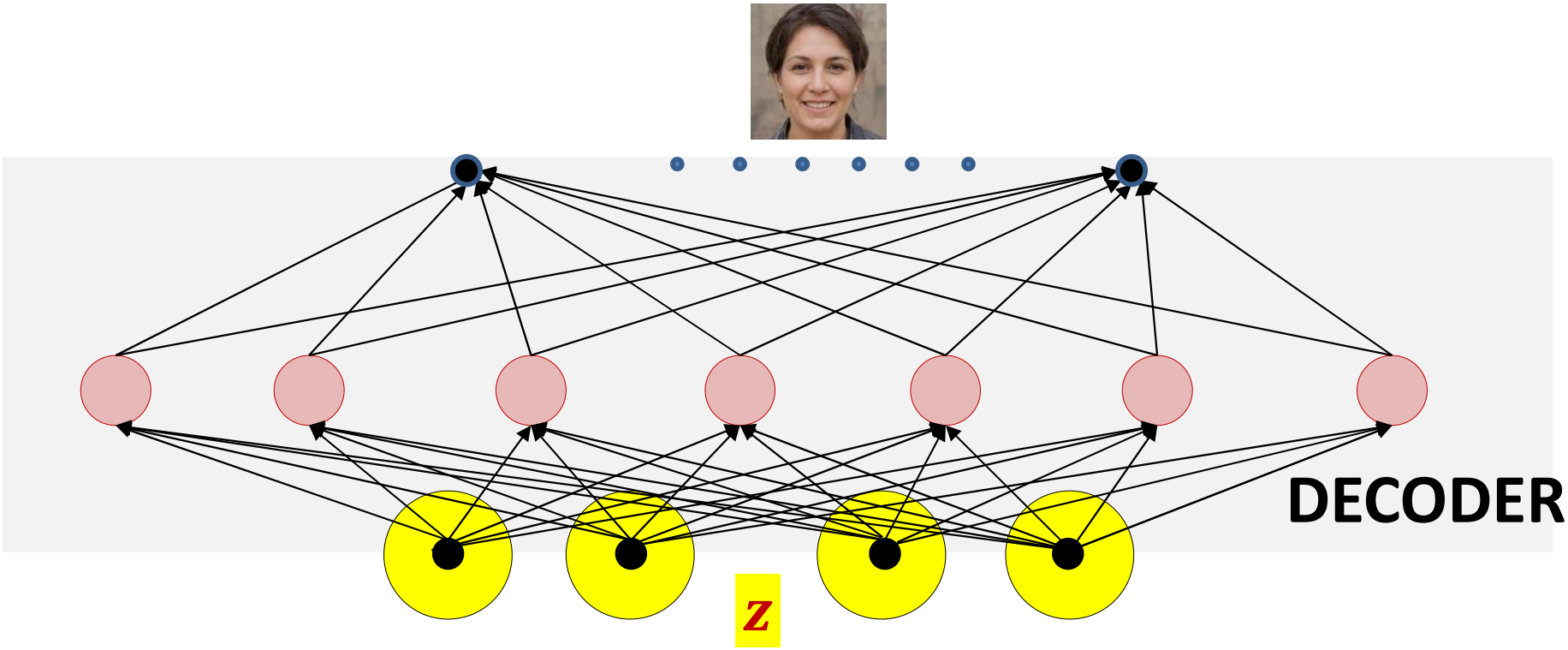
- The decoder represents a source-specific generative *dictionary*
- Exciting it will produce data similar to those from the source!

The AE for generation



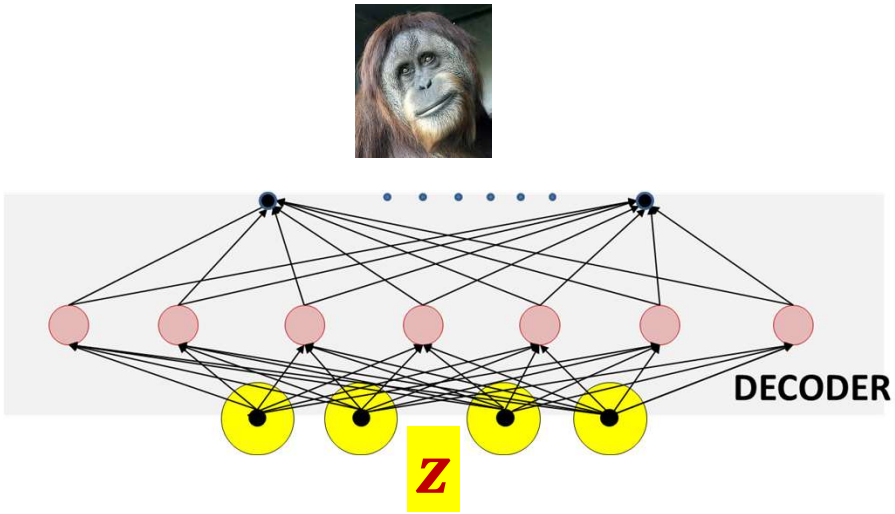
- Train AE with the pictures…

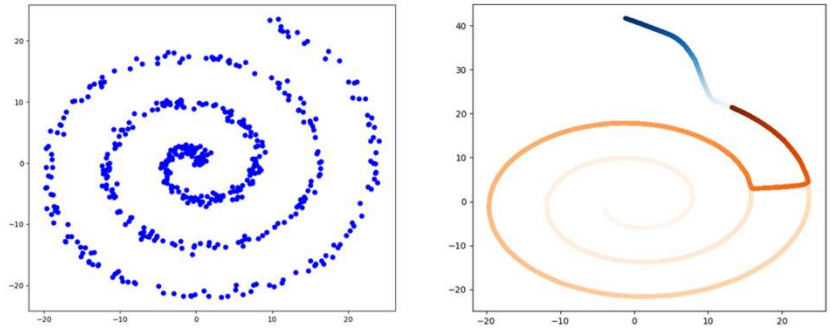
The face dictionary



- The decoder can now be used to generate instances from the “faces” manifold

The problem with AEs





- Improper choice of input to the decoder can result in incorrect generation
- How do we know *what* inputs are reasonable for the decoder?
- Solution : only choose input (z's) that are typical of the class
  - I.e. drawn from the distribution of z's for faces
  - But what is this distribution?

Poll 1 (@, @)

- The decoder in an AE can only generate data on a low-dimensional surface/manifold of the space
  - True
  - False

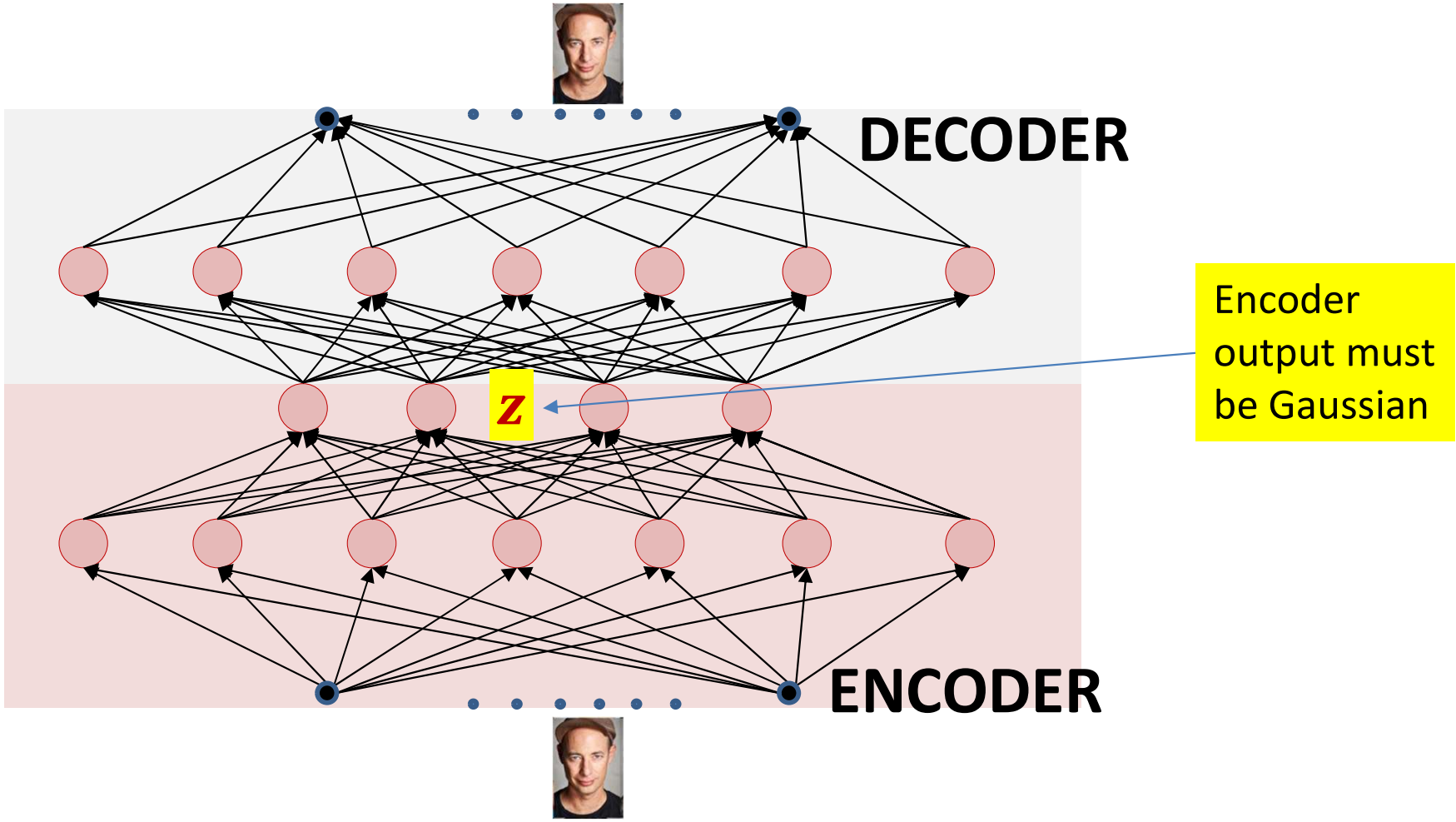
- What is true of the dimensionality of this manifold
  - It cannot be predicted and can be anything
  - It is no greater than the dimensionality of the latent representation that is input to the decoder
  - It will be the same as the input to the encoder

Poll 1

- The decoder in an AE can only generate data on a low-dimensional surface/manifold of the space
  - **True**
  - False

- What is true of the dimensionality of this manifold
  - It cannot be predicted and can be anything
  - **It is no greater than the dimensionality of the latent representation that is input to the decoder**
  - It will be the same as the input to the encoder

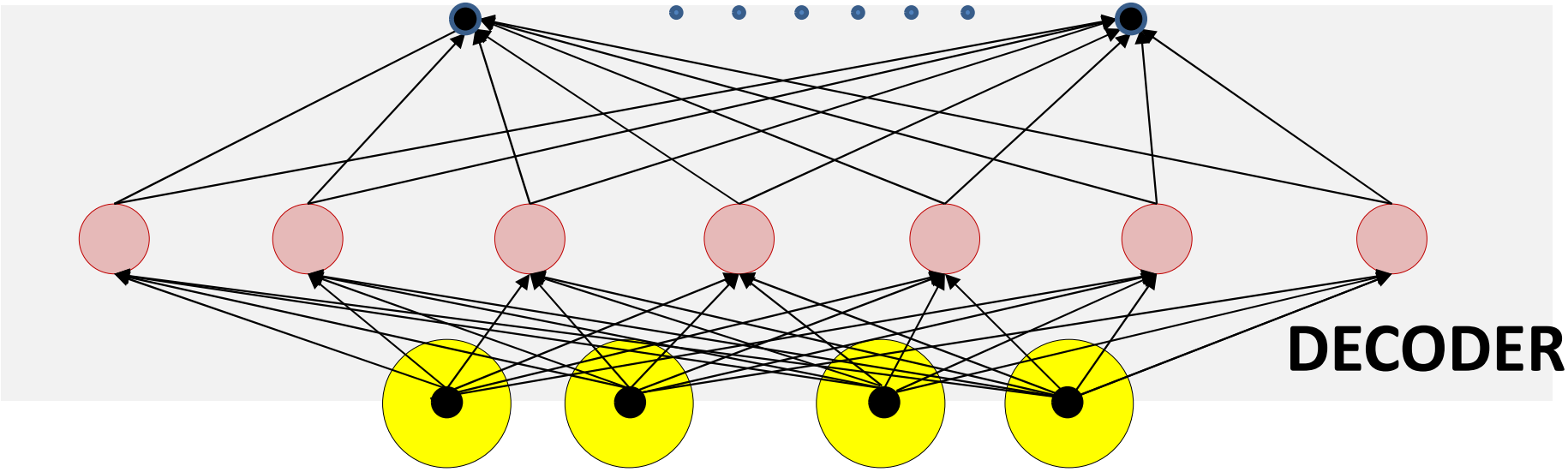
Impose distribution on z



- When training the autoencoder, explicitly impose the constraint that the hidden representation z must follow a specific distribution
  - E.g $P(z)$ is standard Gaussian $(N(0, I))$

Generation

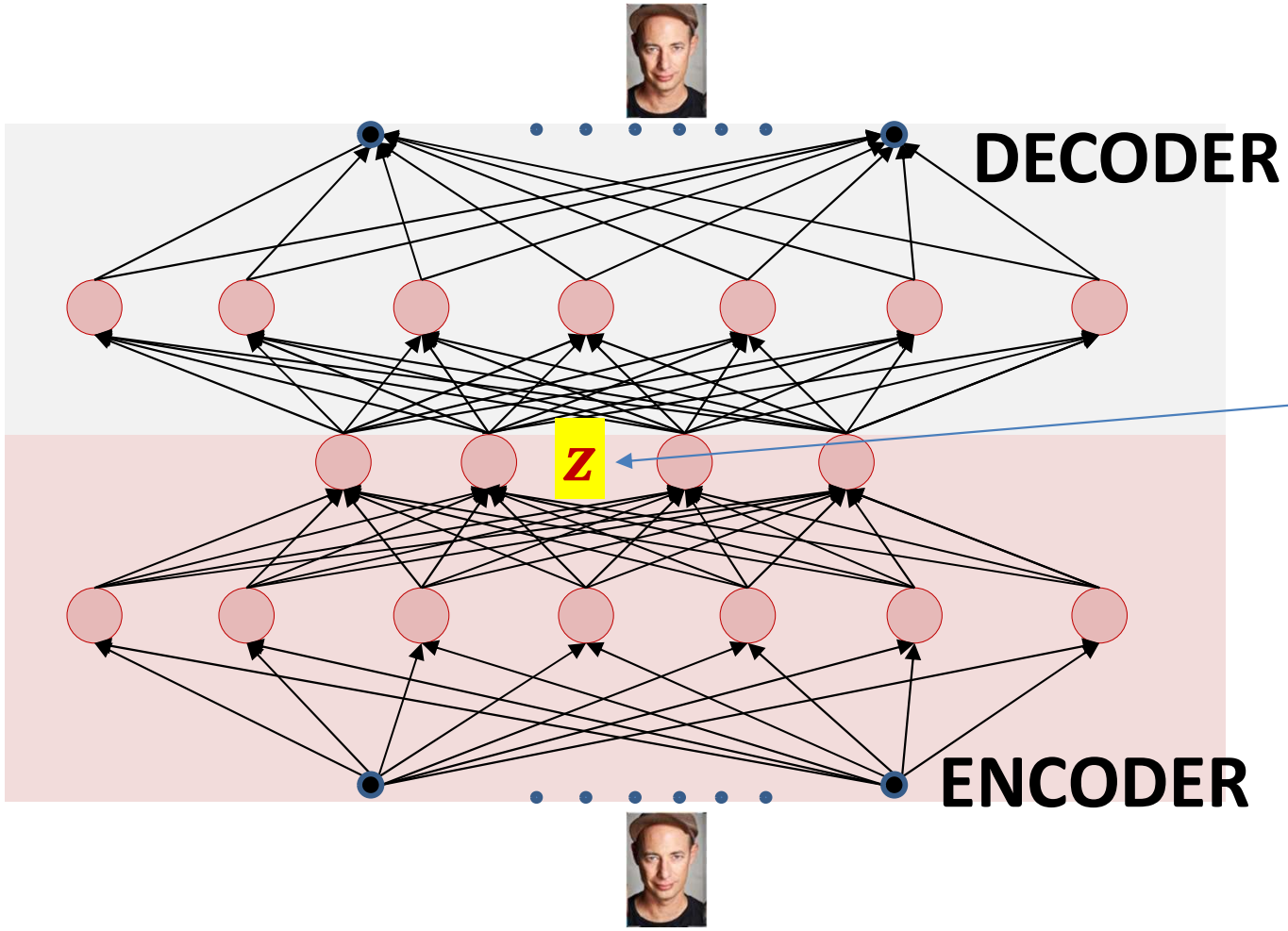




$z \sim N(0, I)$

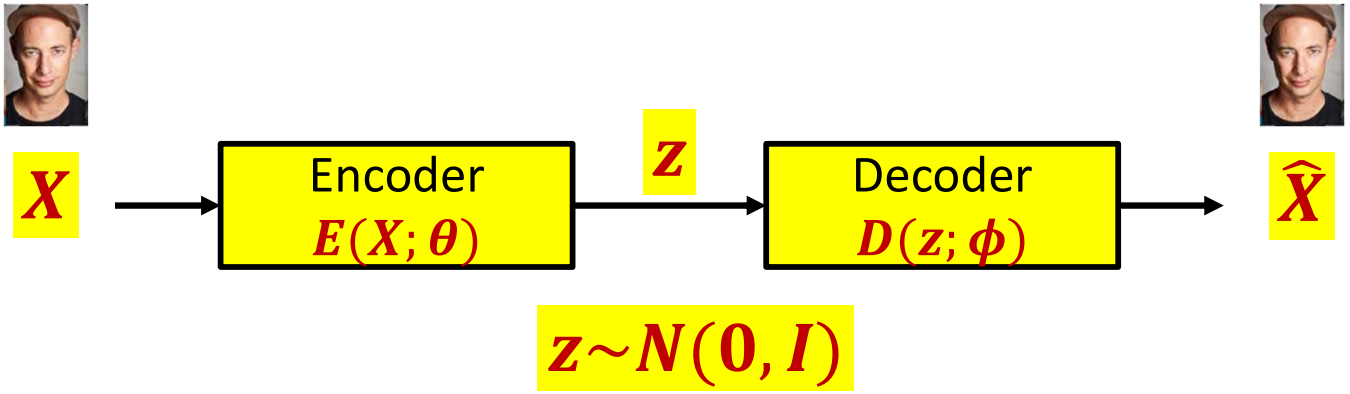
- To generate novel values, sample z from the prescribed distribution ($N(0, I)$)
- If the network is properly trained, and z is properly sampled, the output should be a reasonable generation
  - E.g. of a face

How to train the model



- **Problem:** How does one train an AE to ensure that the hidden representation z has a specific distribution, e.g. $N(0, I)$

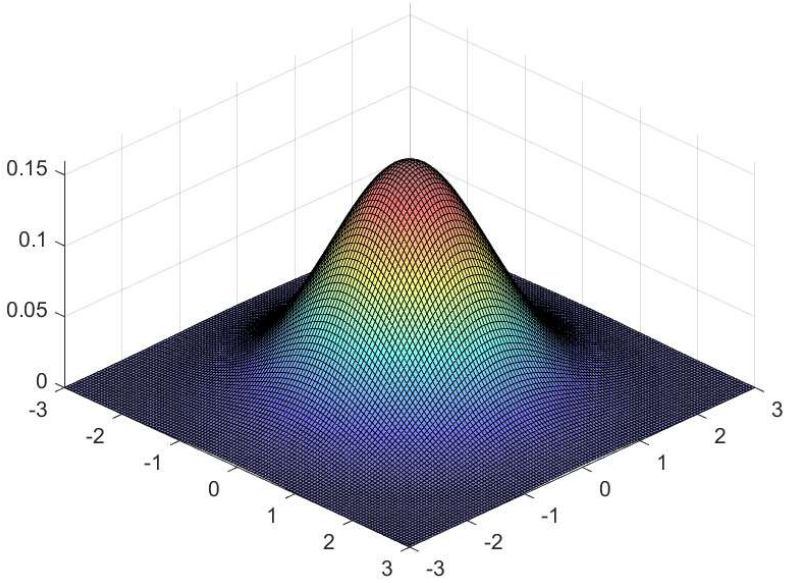
How to train the model

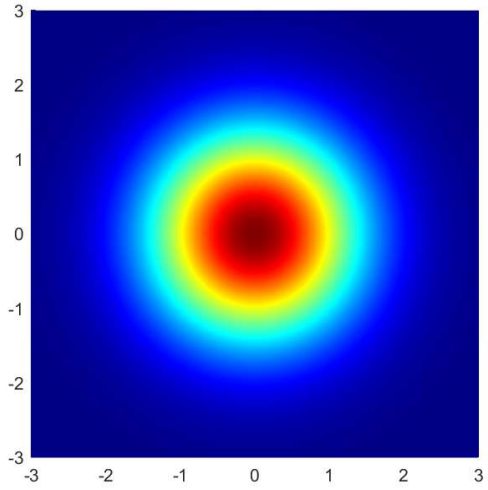


- **Problem:** How does one train an AE to ensure that the hidden representation z has a specific distribution, e.g. $P(z) = N(0, I)$

- Encoder and decoder may have arbitrarily complex structure and their own parameters $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$

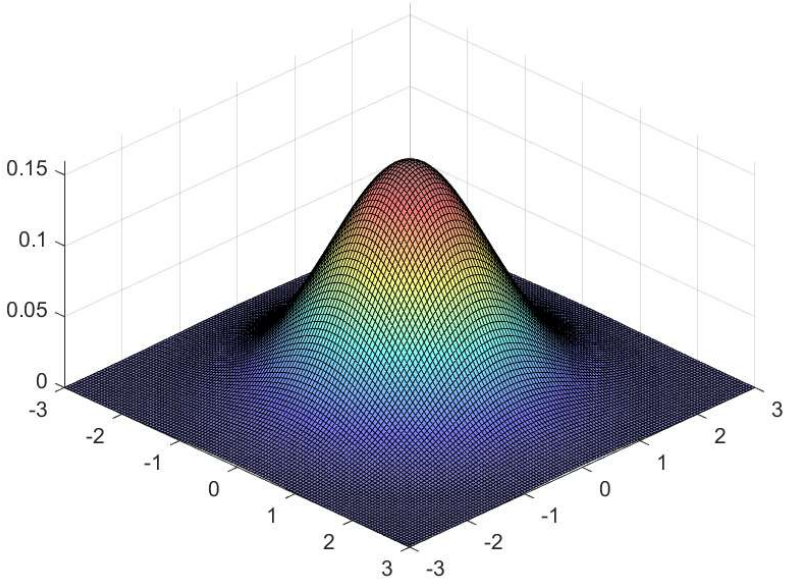
A property of isotropic Gaussians

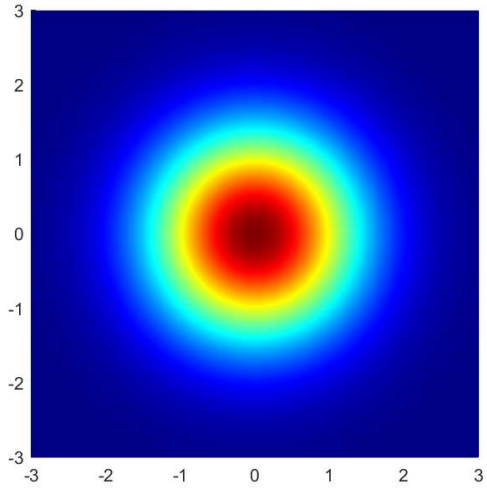




- $P(Z) = N(0, I)$
- The distribution is perfectly symmetric in every direction
  - The different variables (subvectors Z_1 and Z_2 of Z) are independent!
  - $P(Z_1, Z_2) = P(Z_1)P(Z_2)$
  - Each individually will also be isotropic: $P(Z_1) = N(0, I)$

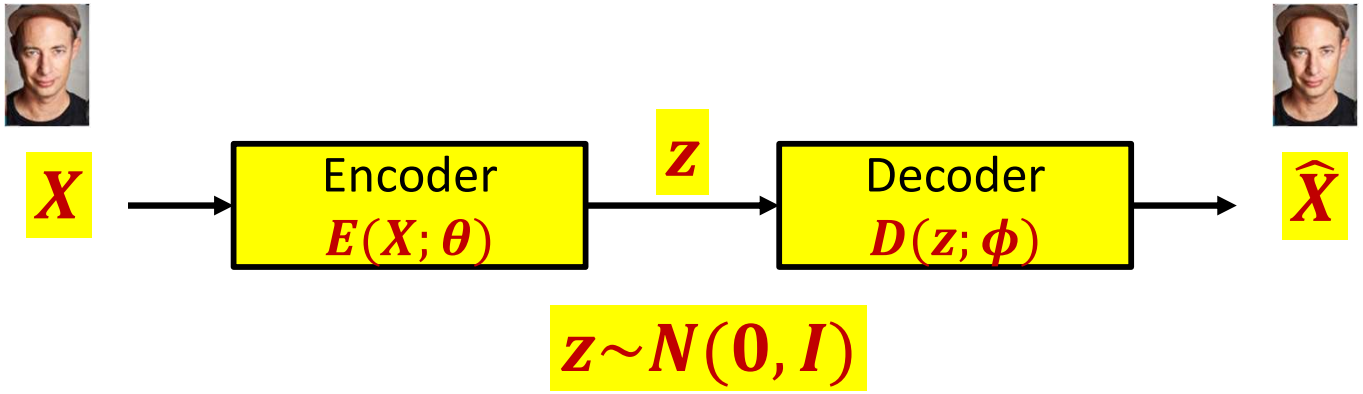
Log likelihood on an isotropic Gaussian





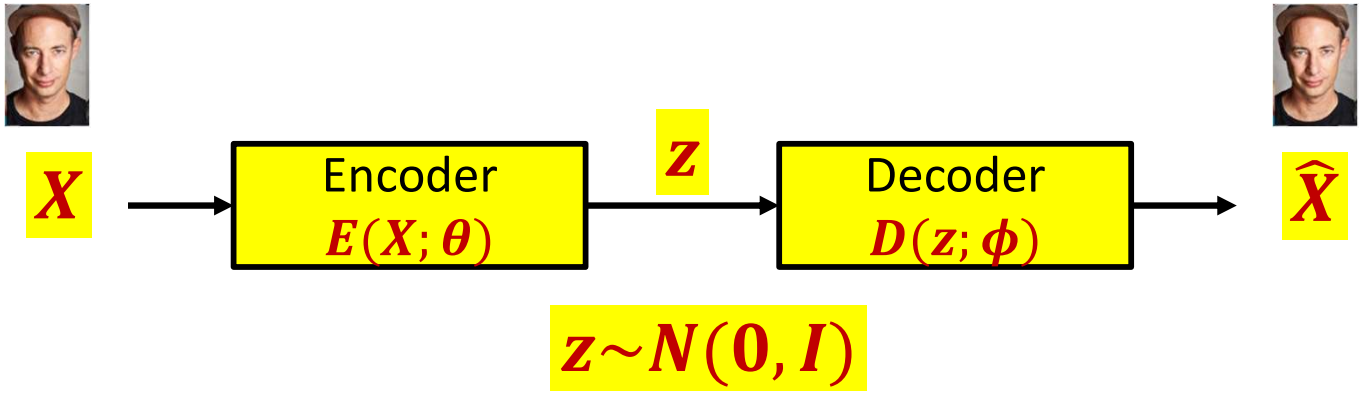
- $P(Z) = \frac{1}{\sqrt{(2\pi)^d}} \exp\left(-0.5|Z|^2\right)$
- $\log P(Z) = -0.5d \log 2\pi - 0.5|Z|^2$
- $-\log P(Z) = 0.5d \log 2\pi + 0.5|Z|^2$

Training with statistical constraints



- Minimize the error between X and $\hat{X}$
- Minimize the KL divergence between the distribution of z and the standard Gaussian $N(0, I)$
- Minimize the negative log likelihood of z as computed from a standard Gaussian

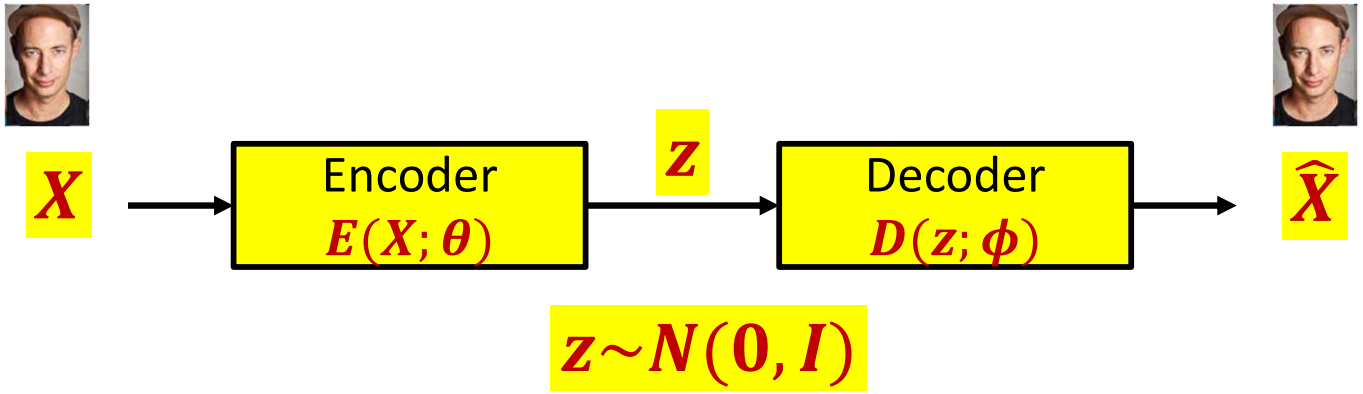
Training with statistical constraints



- $$\min \sum_{z} -\log N(z; 0, I) = \min 0.5 \sum_{X} |z|^2$$
- $$= \min \sum_{X} |E(X; \theta)|^2$$

- Minimize the negative log likelihood of z as computed from a standard Gaussian

Training with statistical constraints



$$\min_{\theta,\phi} \sum_{X} \left|X - \hat{X}\right|^2 + \lambda \left| E(X,\theta) \right|^2$$

Poll 2 (@, @)

- A regular AE trained to also minimize the length of the hidden (latent) representation implicitly assumes an isotropic Gaussian distribution on the latent representation
  - True
  - False

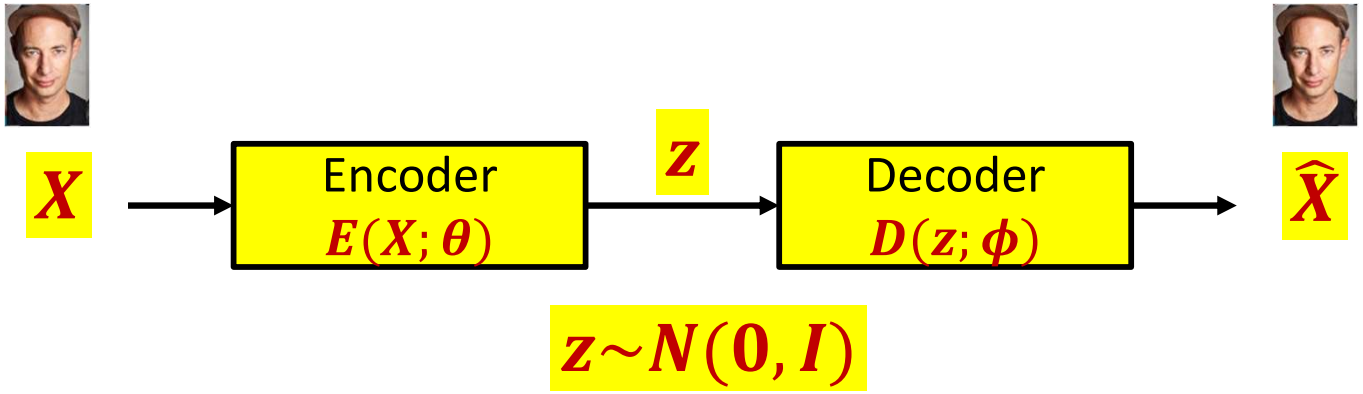
- The output of an AE trained in this manner is no longer constrained to lie on a low dimensional manifold
  - True
  - False

Poll 2

- A regular AE trained to also minimize the length of the hidden (latent) representation implicitly assumes an isotropic Gaussian distribution on the latent representation
  - **True**
  - False

- The output of an AE trained in this manner is no longer constrained to lie on a low dimensional manifold
  - True
  - **False**

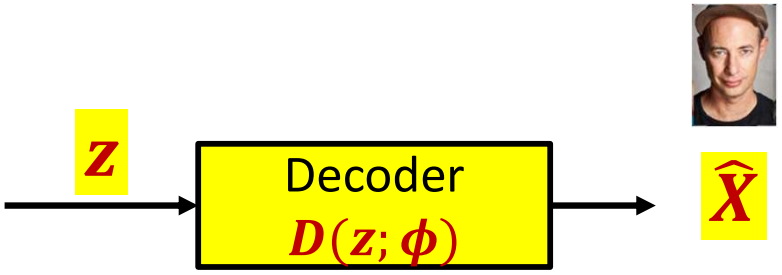
This model does not capture the entire distribution



$$\min_{\theta,\phi} \sum_{X} \left|X - \hat{X}\right|^2 + \lambda \left| E(X,\theta) \right|^2$$

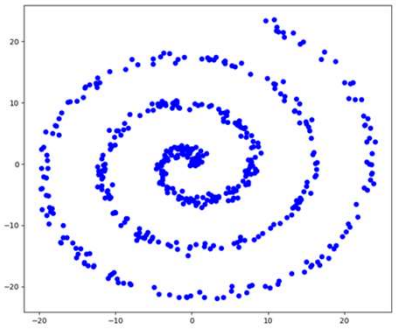
- This simple formulation does not adequately capture the variation in the data
  - The simple ML model will 'push' all training data to simply have very small z, with low variance

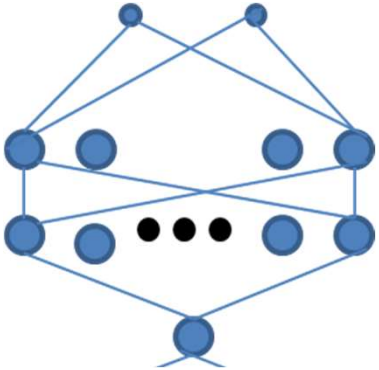
The problem

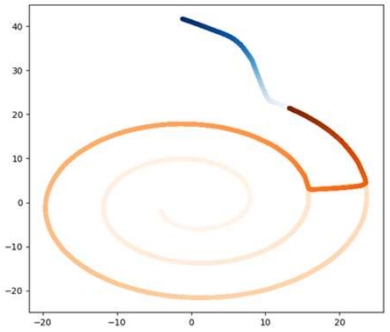


- The generative portion of the model is just the decoder
  - Inputting Gaussian data into it still only produces what the decoder is capable of producing!
    - Secondary problem: We've only taught it to produce a good face for |z| = 0
      - The rest will be garbage…

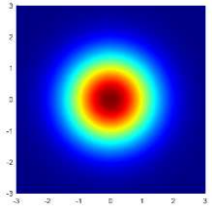
The problem



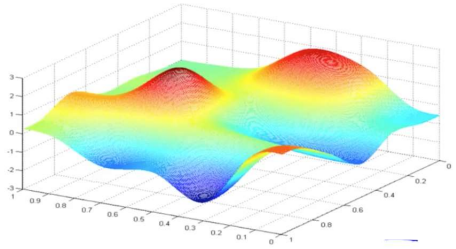




z

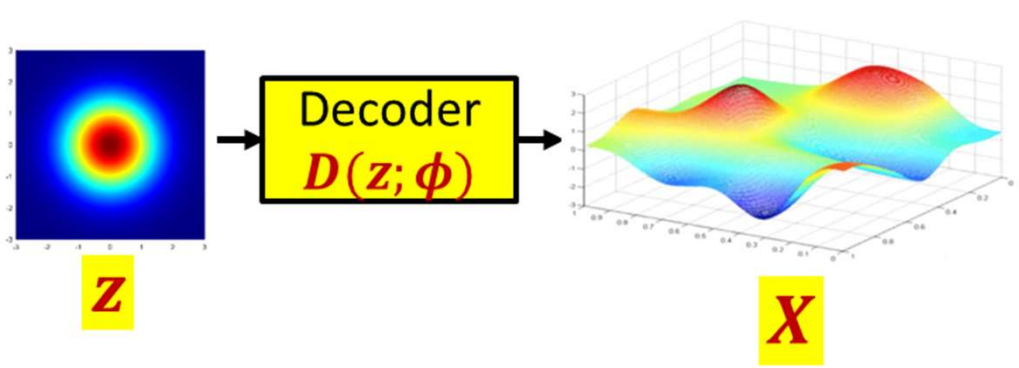


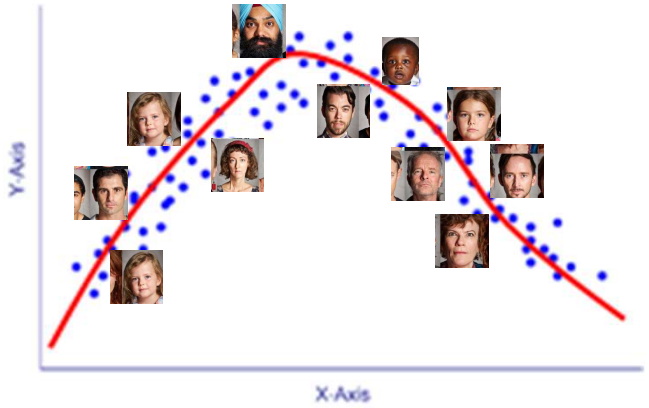
z → Decoder $D(z;\phi)$ → $\hat{X}$



- The decoder can *only* generate data on a low-dimensional manifold of the space
  - Of the same dimension as the input
  - It transforms the planar input space to a curved manifold in the output space
    - and the Gaussian to a non-Gaussian distribution on this manifold

The problem

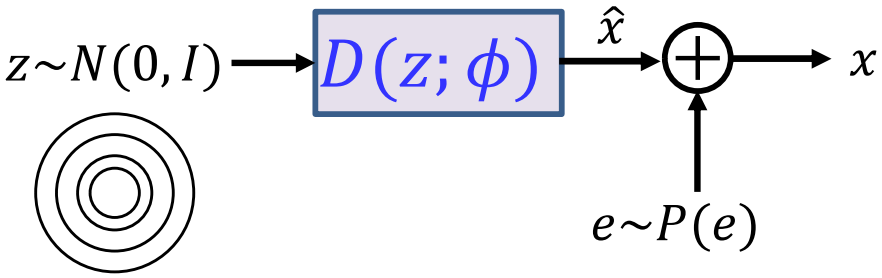


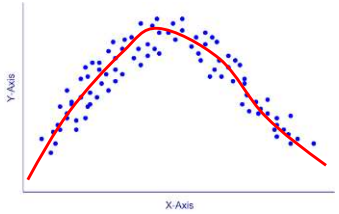


- The actual dimensionality of the data manifold may be (and generally will be) greater than the dimensionality of z
- *Even* if we capture the dimensionality of the principal manifold perfectly, there will almost *always* be some variation off it
  - The actual distribution lies *close* to a curved manifold, but is nonetheless full-dimensional
- Our simple model captures none of the variation off the manifold!

Accounting for the noise

$$x = D(z) + e$$

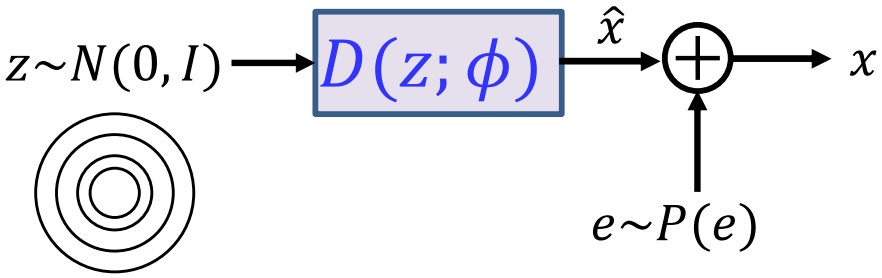


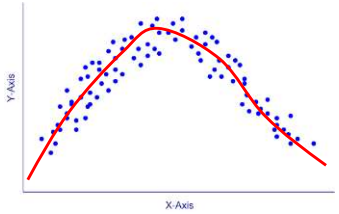


- Assume that the actual data is obtained by adding noise to the output of the decoder
  - The noise accounts for the variation off the manifold
- The model itself must be trained to minimize this noise
  - Ensuring that it has captured the principal manifold most effectively
    - This is the *smallest* amount of noise to be added, such that the distribution at the output of the decoder fully explains the distribution the training data
  - Typical Assumption: The noise is uncorrelated Gaussian $N(0, C)$ (i.e. it has maximum entropy) and is independent of z
    - This is a natural assumption: All other predictable structure would ideally be captured by $D(z;\ \phi)$

Accounting for the noise

$$x = D(z) + e$$





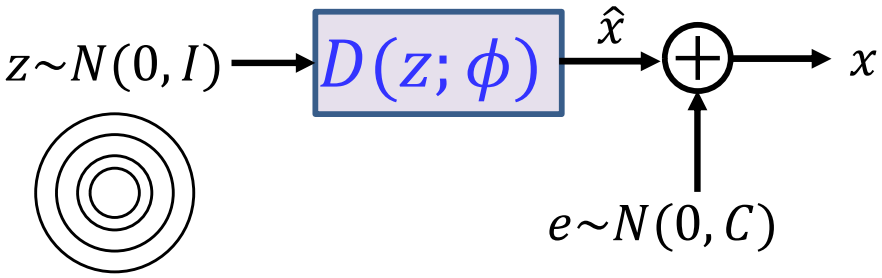
- Assume that the actual data is obtained by adding noise to the output of the decoder
  - The noise accounts for the variation off the manifold

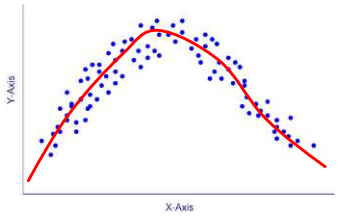
- The model itself must be trained to minimize this noise
  - Ensuring that it has captured the principal manifold most effectively
    - This is the *smallest* amount of noise to be added, such that the distribution at the output of the decoder fully explains the distribution the training data
  - Typical Assumption: The noise is uncorrelated Gaussian $N(0, C)$ (i.e. it has maximum entropy) and is independent of z
    - This is a natural assumption: All other predictable structure would ideally be captured by $D(z;\ \phi)$

- This has a consequence: x is no longer a deterministic outcome of z
  - Instead, it has a distribution : $p(x \mid z) = N(D(z;\ \phi), C)$

The actual model

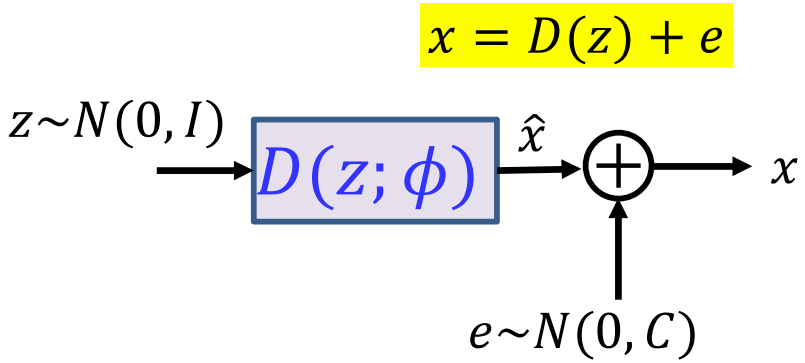
$$x = D(z) + e$$





- The "decoder" is now actually a "generative" model for the data
  - Has a "generative story" for how the data are produced
- A K-dimensional vector z is drawn from a standard K-dimensional Gaussian and passed through the decoder
  - This results in data lying on a K-dimensional non-linear surface in the data space
- Then a full-rank, low-amplitude noise is added to it, to generate the final data
  - The actual data distribution is a fuzzy region around the surface.

The distribution of the data

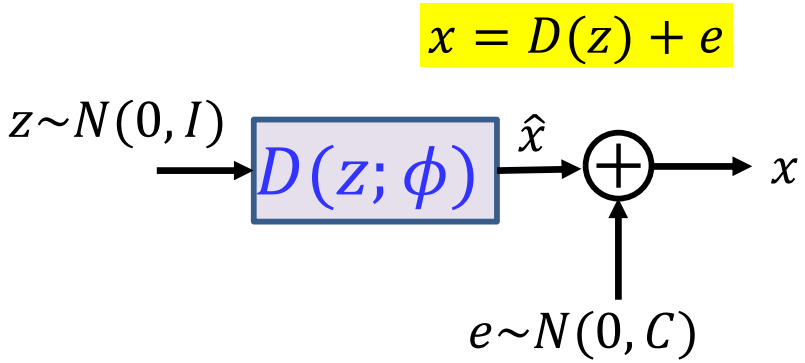


- If the z that went into the decoder to produce any training data instance x is known along with x, the decoder (ϕ) can be estimated

$$\underset{\phi}{\operatorname{argmin}}\, E\left[\|D(z;\phi) - x\|^2\right]$$

Since, by assumption, e is the smallest amount of noise needed for the output of the model to match the training data

The distribution of the data



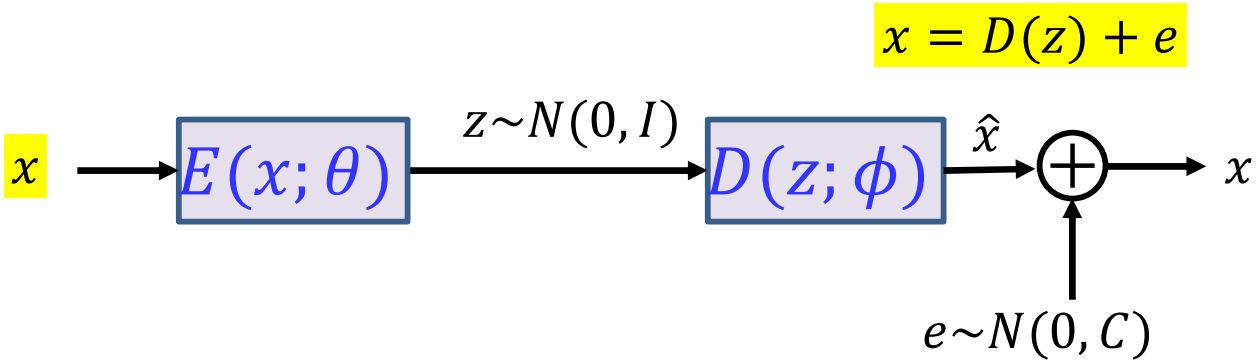
- If the z that went into the decoder to produce any training data instance x is known along with x, the decoder (ϕ) can be estimated

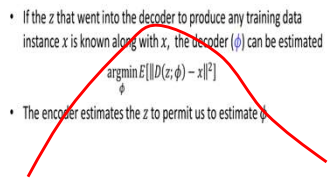
$$\underset{\phi}{\operatorname{argmin}}\, E\left[\|D(z;\phi) - x\|^2\right]$$

Since, by assumption, e is the smallest amount of noise needed for the output of the model to match the training data

Unfortunately, we don't have the z for each x. So, we will estimate it!!

The distribution of the data



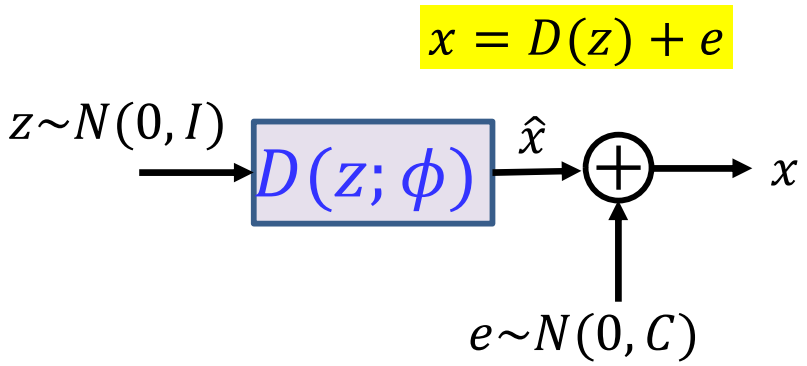


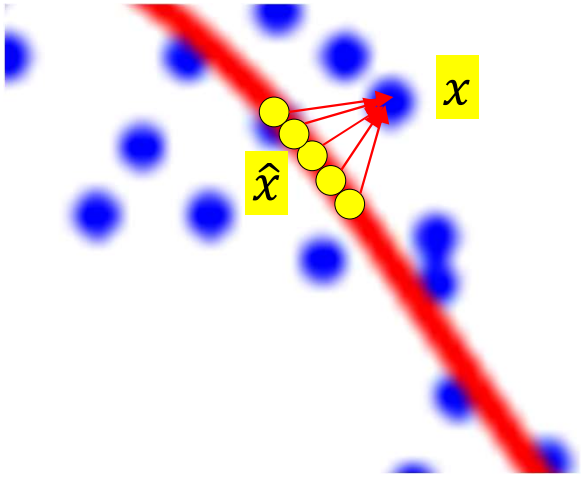
- If the z that went into the decoder to produce any training data instance x is known along with x, the decoder (ϕ) can be estimated

$$\underset{\phi}{\operatorname{argmin}}\, E\left[\|D(z;\phi) - x\|^2\right]$$

- The encoder estimates the z to permit us to estimate ϕ

The distribution of the data





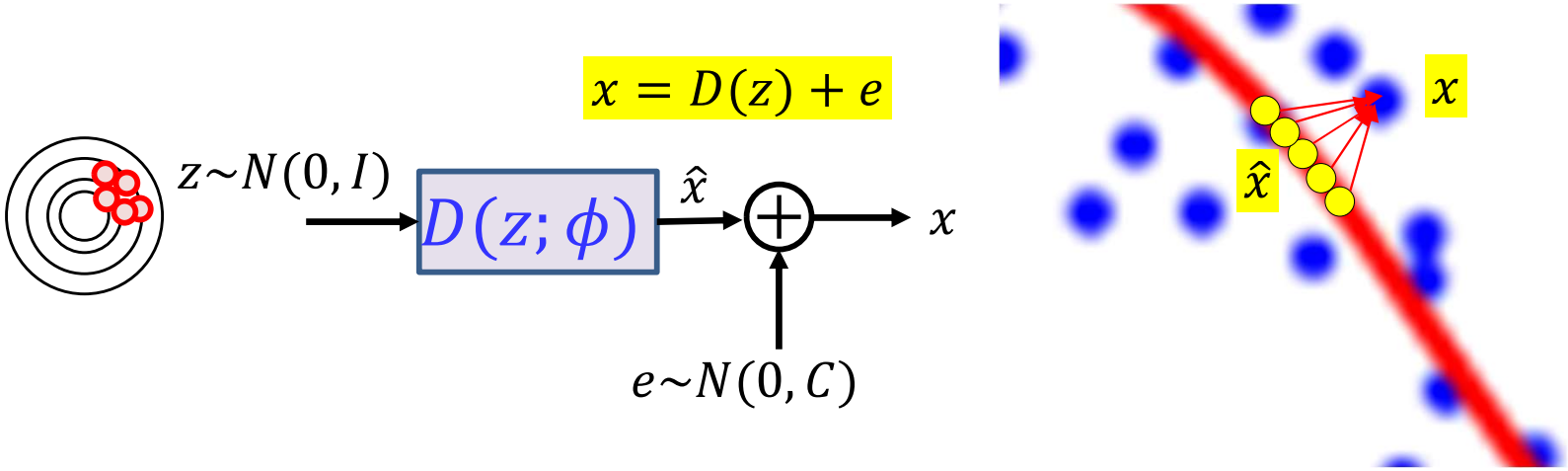
- If the z that went into the decoder to produce any data instance x is known along with x, the decoder (ϕ) can be estimated

$$\underset{\phi}{\operatorname{argmin}}\, E\left[\|D(z;\phi) - x\|^2\right]$$

- The encoder estimates the z to permit us to estimate ϕ

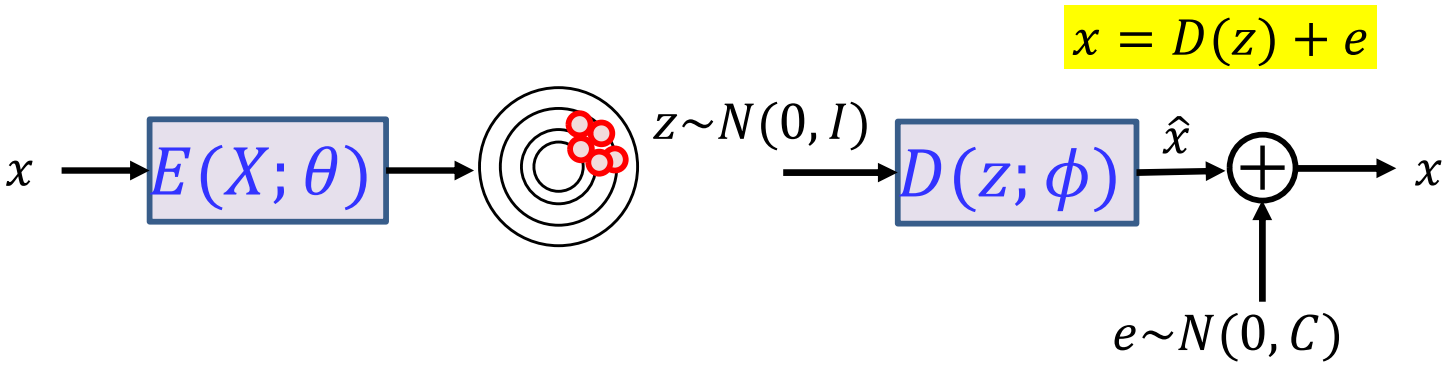
- Problem: Several $\hat{x}$ values exist that could be modified by noise to produce a given x
  - The z that could produce a given x is not unique

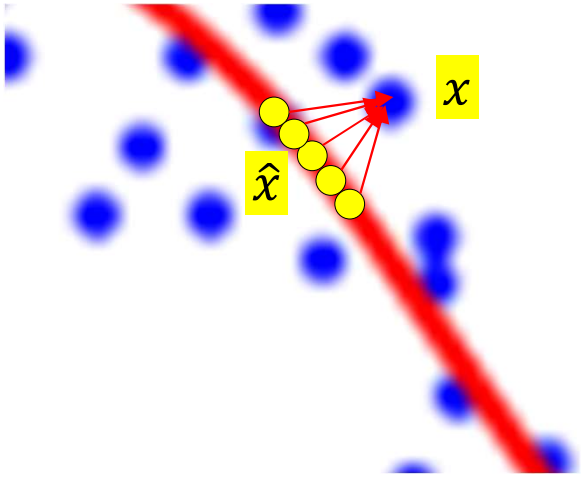
The distribution of the data



- There is an entire *distribution* of zs that could produce a given x
  - z values that correspond to more probable values of noise are more probable
  - The distribution of z for a given x are dependent on that x: $P(z|x)$
- Instead of finding the unique z for any x, we will find the distribution $P(z|x)$

The distribution of the data





- There is an entire *distribution* of zs that could produce a given x
  - z values that correspond to more probable values of noise are more probable
  - The distribution of z for a given x are dependent on that x: $P(z|x)$
- Instead of finding the unique z for any x, we will find the distribution $P(z|x)$

- ***The variational autoencoder***
  - So named because the learning procedure utilizes a variational bound on the likelihood of the data

The variational autoencoder



- The *encoder* computes the *distribution* $P(z|x)$ for any x
- The *decoder* tries to convert the *expected* output for zs drawn from this distribution to x
  - More practically, it tries to convert a typical sample z from $P(z|x)$ to x

The variational autoencoder



- The *encoder* computes the *distribution* $P(z|x)$ for any x
- The *decoder* tries to convert a typical sample z from $P(z|x)$ to x

- **Training the encoder**:
  - Estimate θ to make the zs that can be decoded to $\hat{x}$ values that are closer to x more probable

The variational autoencoder



- The *encoder* computes the *distribution* $P(z|x)$ for any x
- The *decoder* tries to convert a typical sample z from $P(z|x)$ to x

- **Training the encoder**:
  - Estimate θ to make the zs that can be decoded to $\hat{x}$ values that are closer to x more probable

- **Training the decoder**:
  - Estimate $\boldsymbol{\phi}$ to make the noise e between $\hat{x} = D(z)$ and x more probable
    - I.e. make it as small as possible

The variational autoencoder



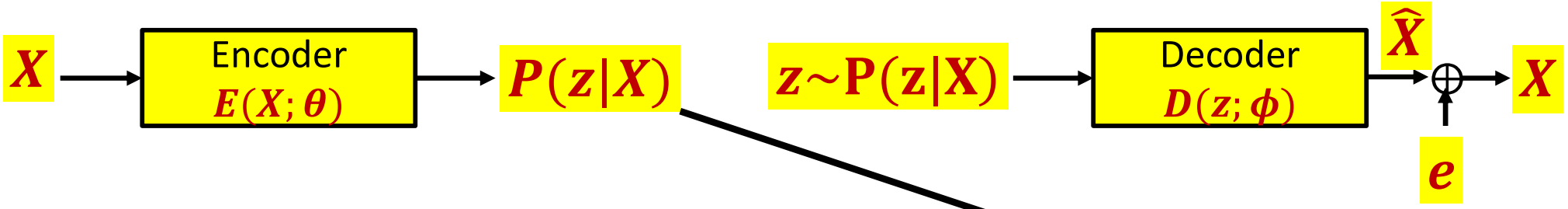
- The *encoder* computes the *distribution* $P(z|x)$ for any x
- The *decoder* tries to convert a typical sample z from $P(z|x)$ to x

- **Training the encoder**:
  - Estimate θ to make the zs that can be decoded to $\hat{x}$ values that are closer to x more probable

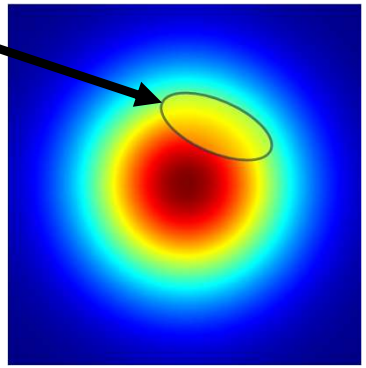
- **Training the decoder**:
  - Estimate $\boldsymbol{\phi}$ to make the noise e between $\hat{x} = D(z)$ and x more probable
    - I.e. make it as small as possible

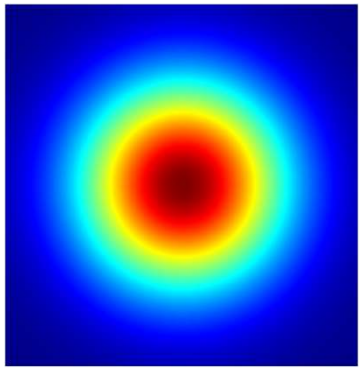
- **Constraint on z:** ?

The variational autoencoder

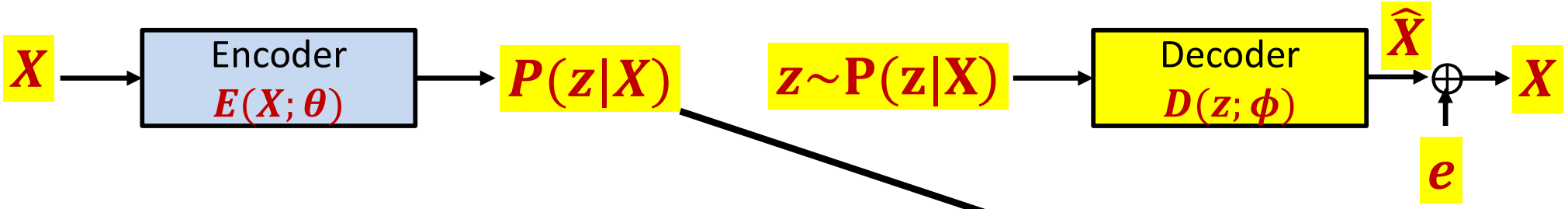


- The *conditional* distribution of z given X is $P(z|X)$
  - Specifies the region of z from which the specific z that produced *this specific* X could have been drawn
- **But the *overall* distribution of z is a standard Gaussian**

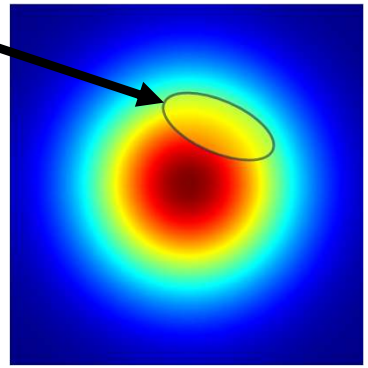


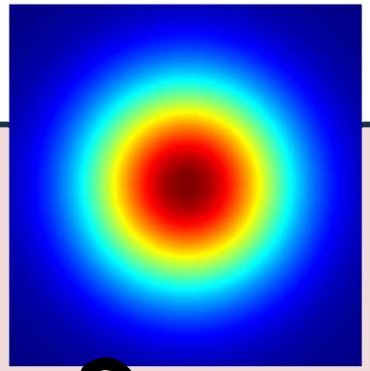


The variational autoencoder



- The *conditional* distribution of z given X is $P(z|X)$
  - Specifies the region of z from which the specific z that produced *this specific* X could have been drawn

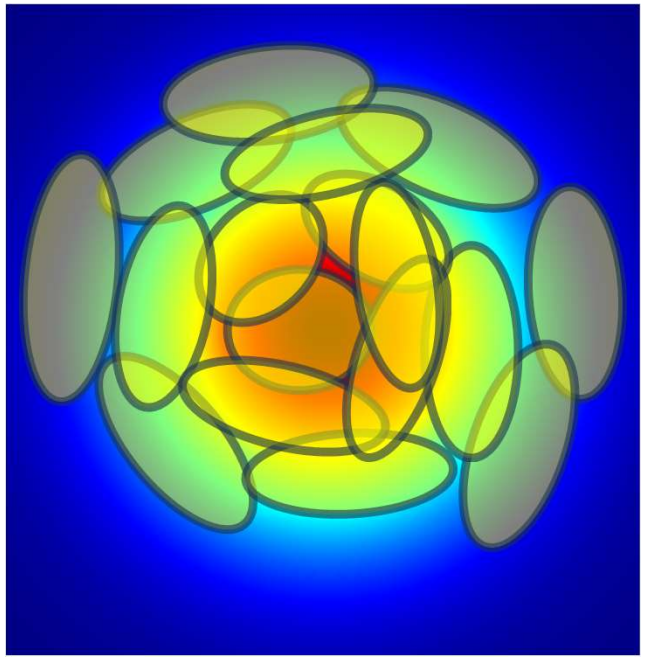




- **But the *overall* distribution of z is a standard Gaussian**

What does this mean?

What it means…



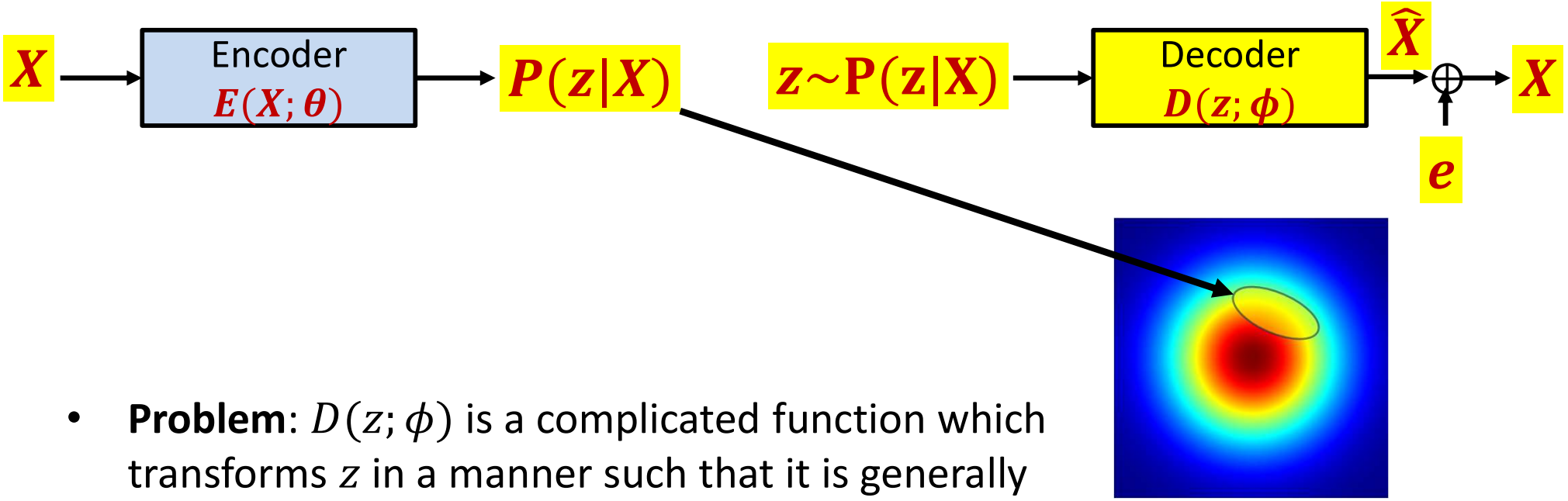
- The *combined* posterior distribution $P(\boldsymbol{z}|\boldsymbol{X})$ over *all* possible values of $\boldsymbol{X}$ is a standard normal distribution

- According to the model:

$$\int_{-\infty}^{\infty} P(\boldsymbol{X})P(\boldsymbol{z}|\boldsymbol{X})d\boldsymbol{X} = N(0, I)$$

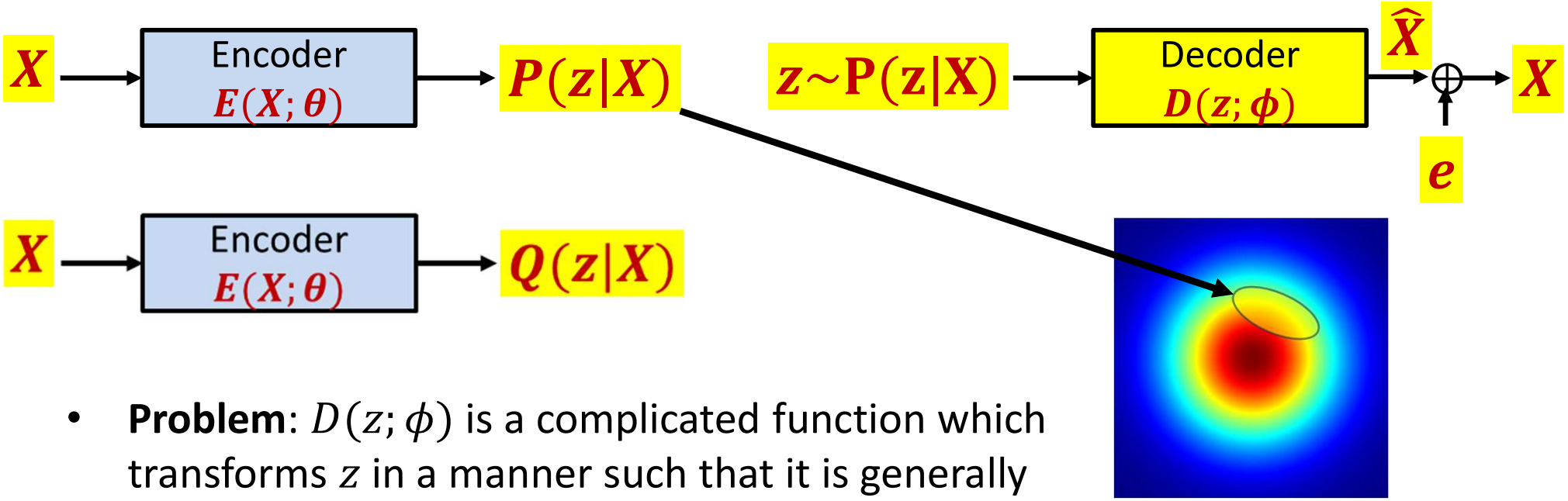
- I.e. we must train the model such that the combined posterior over all $\boldsymbol{X}$s is a standard normal

A challenge : P(z|X) is unfriendly



- **Problem**: $D(z;\phi)$ is a complicated function which transforms z in a manner such that it is generally not possible to characterize $P(z|X)$
  - The Encoder cannot give you the *actual* $P(z|X)$

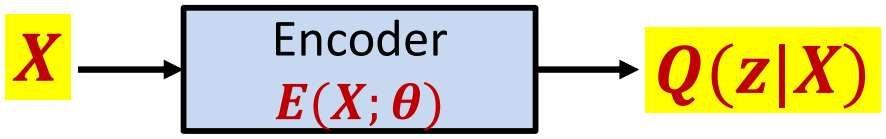
A challenge : P(z|X) is unfriendly

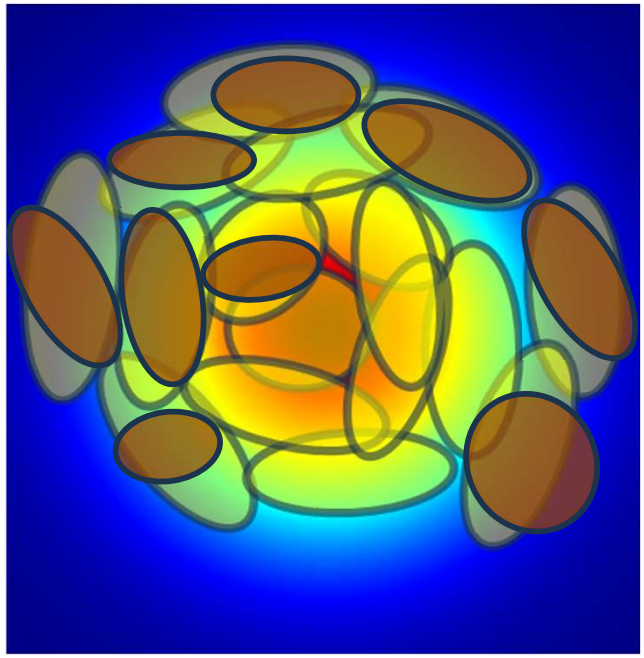


- **Problem**: $D(z;\phi)$ is a complicated function which transforms z in a manner such that it is generally not possible to characterize $P(z|X)$
  - The Encoder cannot give you the *actual* $P(z|X)$

- **Solution**: Instead of deriving the true $P(z|X)$ the Encoder computes an estimate $Q(z|X)$
  - Training must also ensure that $Q(z|X)$ is as close to $P(z|X)$ as possible

Learning the Encoder



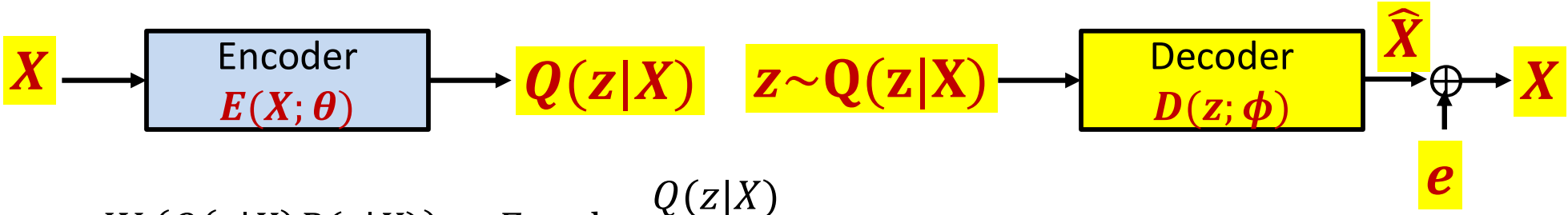


- The encoder actually computes an ***estimate*** $Q(z|X)$ of the actual posterior $P(z|X)$ (in whichever way we do it) for each X
- We must minimize the divergence between the estimate $Q(z|X)$ and the *true* posterior $P(z|X)$ (over all X)

$$= E_X \, KL(Q(Z|X), P(Z|X))$$

- This is intractable in general because $P(Z|X)$ is unknown

Bayes and the Decoder to the rescue



$$KL(Q(z|X)P(z|X)) = E_{z\sim Q}\log\frac{Q(z|X)}{P(z|X)}$$

$$= E_{z\sim Q}\log Q(z|X) - E_{z\sim Q}\log P(z|X)$$

$$= E_{z\sim Q}\log Q(z|X) - E_{z\sim Q}\log\frac{P(z)P(X|z)}{P(X)}$$

$$= E_{z\sim Q}\log Q(z|X) - E_{z\sim Q}\log P(z) - E_{z\sim Q}\log P(X|z) + E_{z\sim Q}\log P(X)$$

$$= KL\big(Q(z|X), P(z)\big) - E_{z\sim Q}\log P(X|z) + E_{z\sim Q}\log P(X)$$

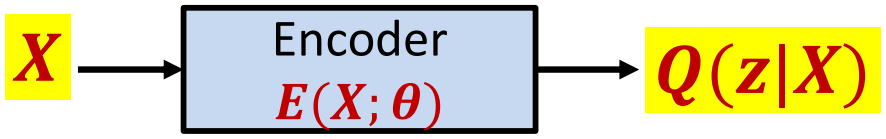
- $Q(z|X)$ is a function of φ. Minimizing the loss w.r.t. φ we get

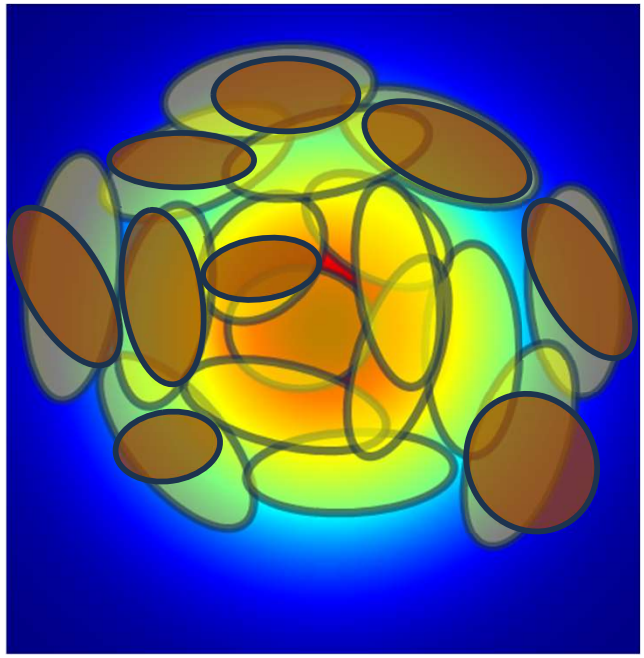
$$\varphi^* = \underset{\varphi}{\operatorname{argmin}}\, KL(Q(z|X)P(z|X))$$

$$= \underset{\varphi}{\operatorname{argmin}}\, KL\big(Q(z|X), P(z)\big) - E_{z\sim Q}\log P(X|z)$$

NLL of Decoder

The *variational* autoencoder



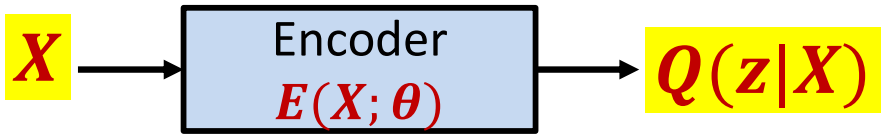


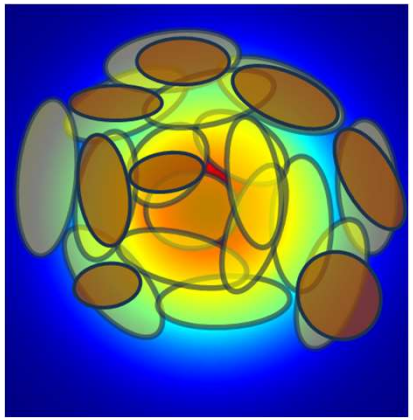
- The modification gives us the simpler optimization

$$= E_X \, KL(Q(\boldsymbol{z}|\boldsymbol{X}), N(0, I))$$

- **We must minimize the *expected divergence between the distribution $Q(z|X)$ modelled by the encoder and the standard normal $N(0, I)$ to get a good approximation of $P(z|X)$***

The variational Encoder



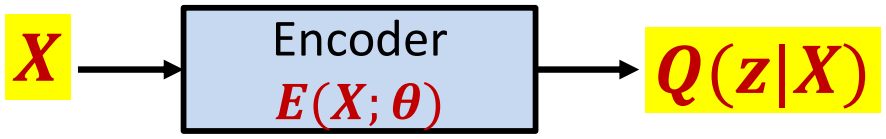


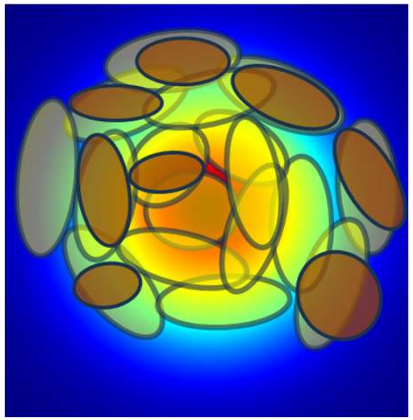
- We must train the encoder to minimize the *expected KL divergence between* $Q(\boldsymbol{z}|\boldsymbol{X})$ *and the standard normal* $N(0, I)$
- Empirically (since we perform Empirical Risk Minimization)

$$E_{\boldsymbol{X}} KL(Q(\boldsymbol{z}|\boldsymbol{X}), N(0, I)) \approx \frac{1}{N} \sum_{\boldsymbol{X}} KL(Q(\boldsymbol{z}|\boldsymbol{X}), N(0, I))$$

- We must train the encoder to minimize the **average** KL divergence between the posterior $Q(\boldsymbol{z}|\boldsymbol{X})$ and the standard normal over the training data

The variational Encoder



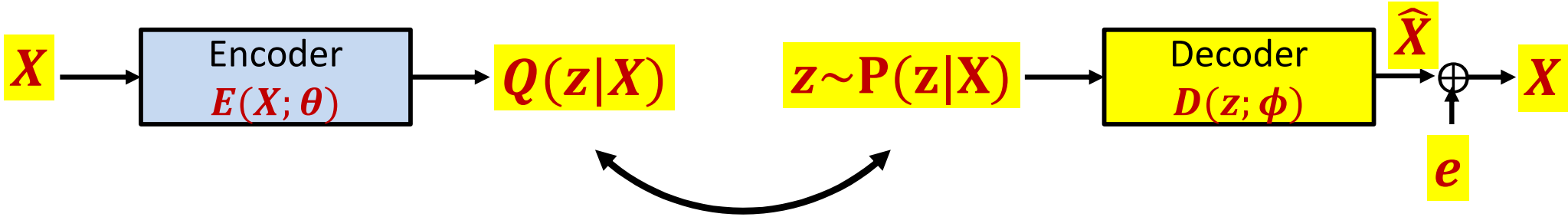


- We must train the encoder to minimize the *expected KL divergence between* $Q(\boldsymbol{z}|\boldsymbol{X})$ *and the standard normal* $N(0, I)$
- Empirically (since we perform Empirical Risk Minimization)

$$E_{\boldsymbol{X}} KL(Q(\boldsymbol{z}|\boldsymbol{X}), N(0, I)) \approx \frac{1}{N} \sum_{\boldsymbol{X}} KL(Q(\boldsymbol{z}|\boldsymbol{X}), N(0, I))$$

- We must train the encoder to minimize the **average** KL divergence between the posterior $Q(\boldsymbol{z}|\boldsymbol{X})$ and the standard normal over the training data
- For this, the encoder must model the *distribution,* $Q(\boldsymbol{z}|\boldsymbol{X})$
  - It is now computing an entire distribution rather than a single value of $\boldsymbol{z}$

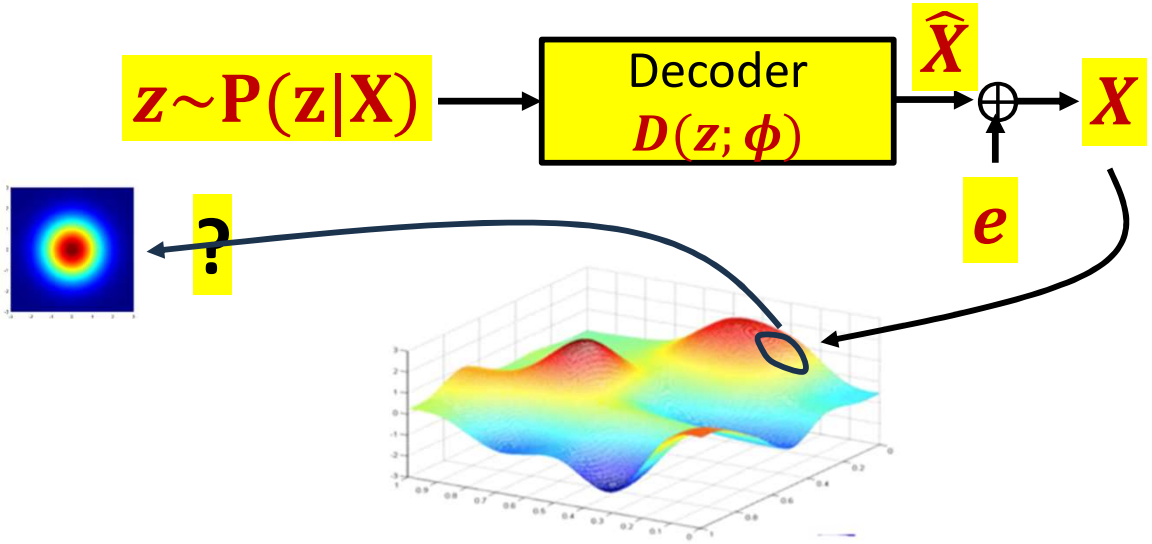
The problem for the encoder lies in the decoder…

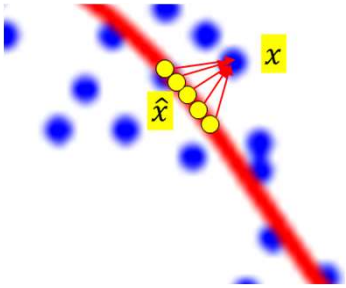


- Recall that $Q(z|X)$ must approximate $P(z|X)$ as closely as possible
  - The distribution of z that could generate any X
  - What does this look like?

- The decoder that converts z to $\hat{X}$ is highly nonlinear
  - Difficult to characterize the distribution of the z values that could produce a given X

The problem for the encoder lies in the decoder…

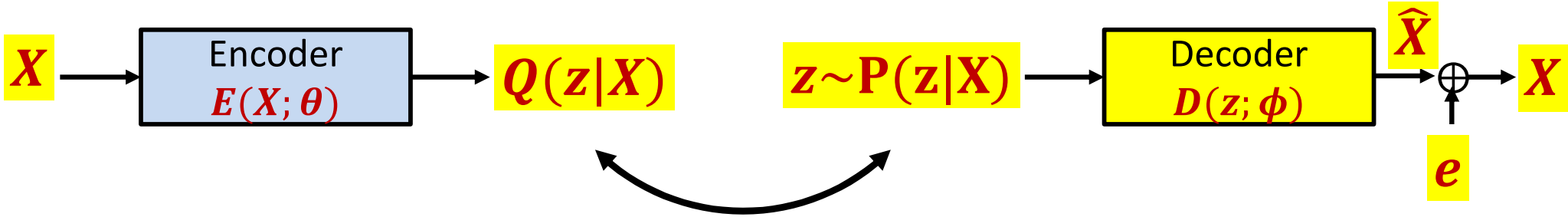




- An output $\mathbf{X}$ could have come from an entire set/distribution of $\hat{\boldsymbol{X}}$ values (since we don't know the exact $\boldsymbol{e}$ that was added to $\hat{\boldsymbol{X}}$)
  - In fact $P(\hat{\boldsymbol{X}}|\boldsymbol{X})$ is Gaussian because $\boldsymbol{e}$ is Gaussian

- But because the $\hat{\boldsymbol{X}}$ surface is uneven and wickedly curved, we cannot determine the corresponding set of $\boldsymbol{z}$ values
  - I.e. cannot characterize the set of $\boldsymbol{z}$ values that resulted in the set of $\hat{\boldsymbol{X}}$ values that could have produced $\boldsymbol{X}$

The problem for the encoder lies in the decoder…

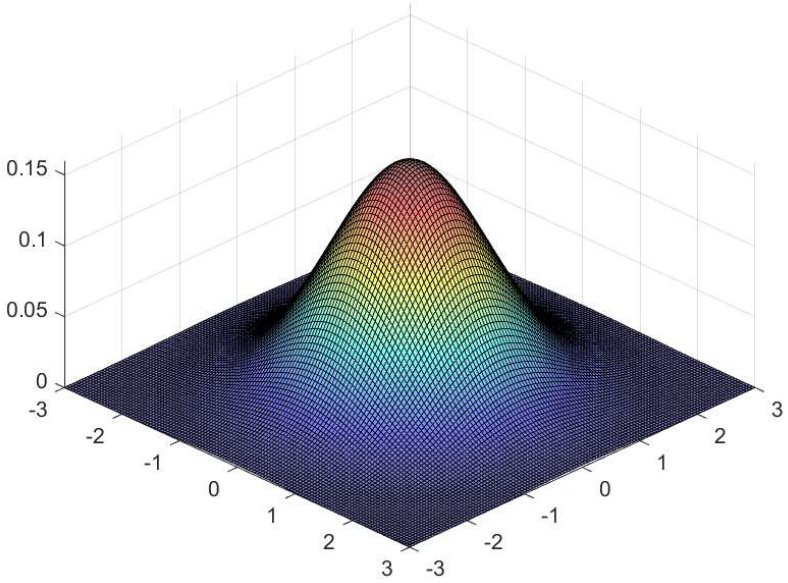


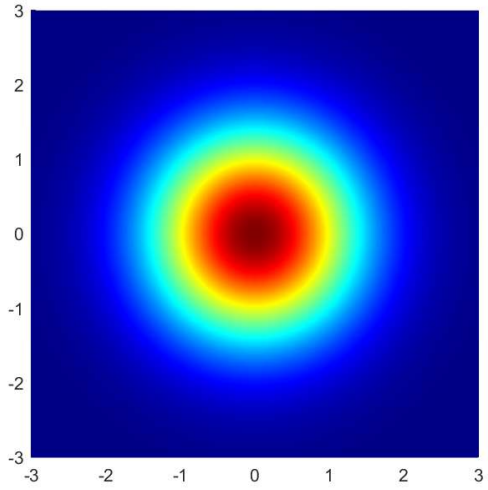
- Recall that $Q(z|X)$ must approximate $P(z|X)$ as closely as possible
  - The distribution of z that could generate any X
  - What does this look like?

- The decoder that converts $\boldsymbol{z}$ to $\hat{\boldsymbol{X}}$ is highly nonlinear
  - Difficult to characterize the distribution of the $\boldsymbol{z}$ values that could produce a given $\boldsymbol{X}$

- Though $\mathrm{P}(\hat{\boldsymbol{X}}|\boldsymbol{X})$ is Gaussian (since $\boldsymbol{e}$ is Gaussian), going back through the decoder, $\boldsymbol{P}(\boldsymbol{z}|\boldsymbol{X})$ is generally not characterizable!
  - We need to make some simplifying assumptions

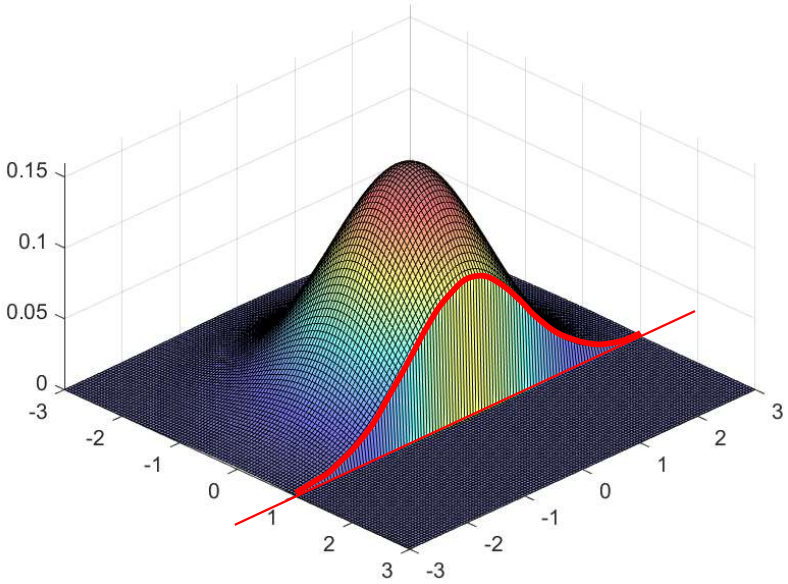
Z is an isotropic Gaussian

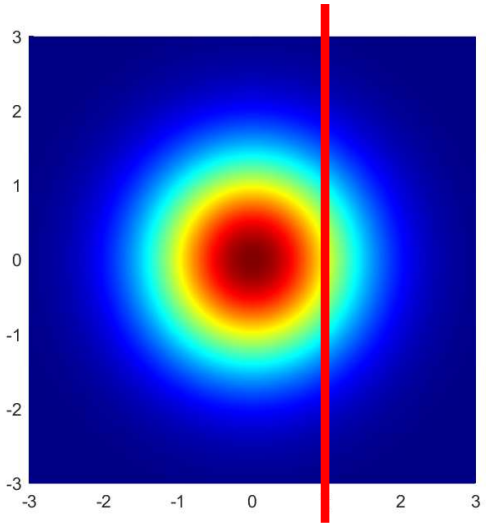




- $P(Z) = N(0, I)$

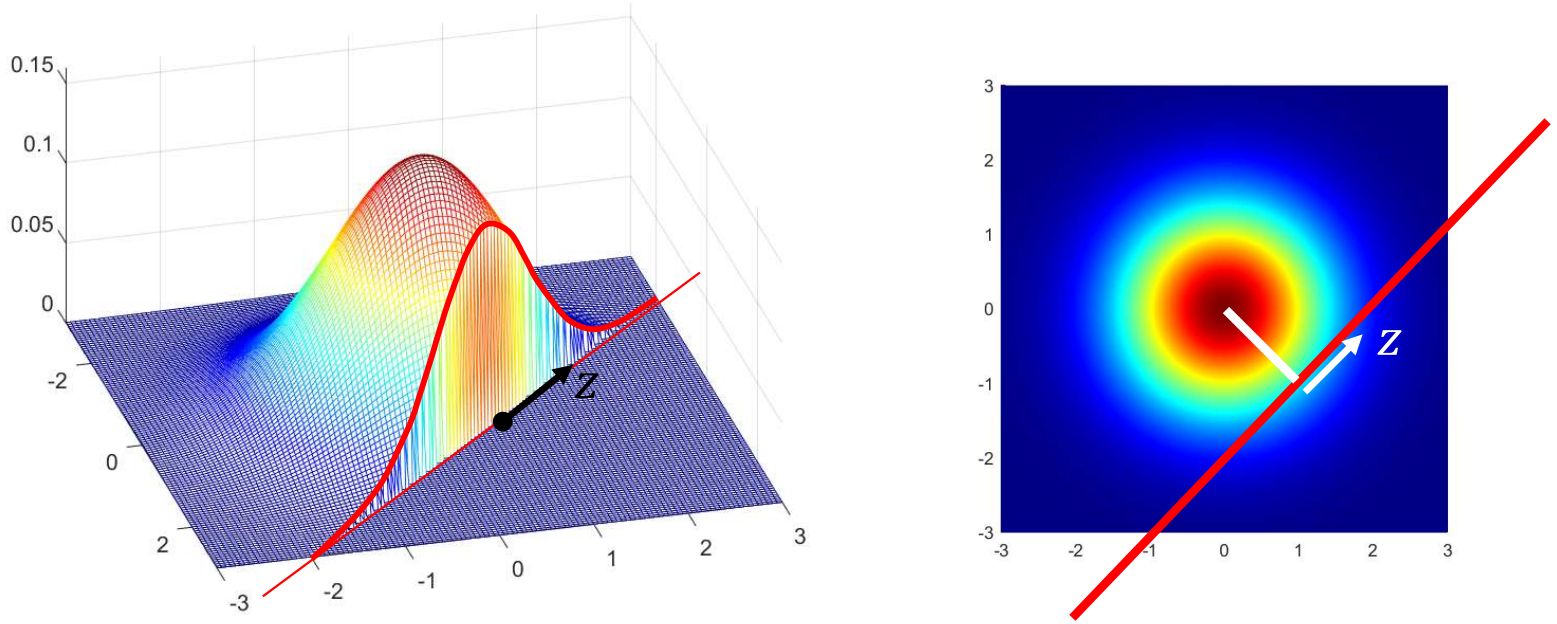
A property of isotropic Gaussians





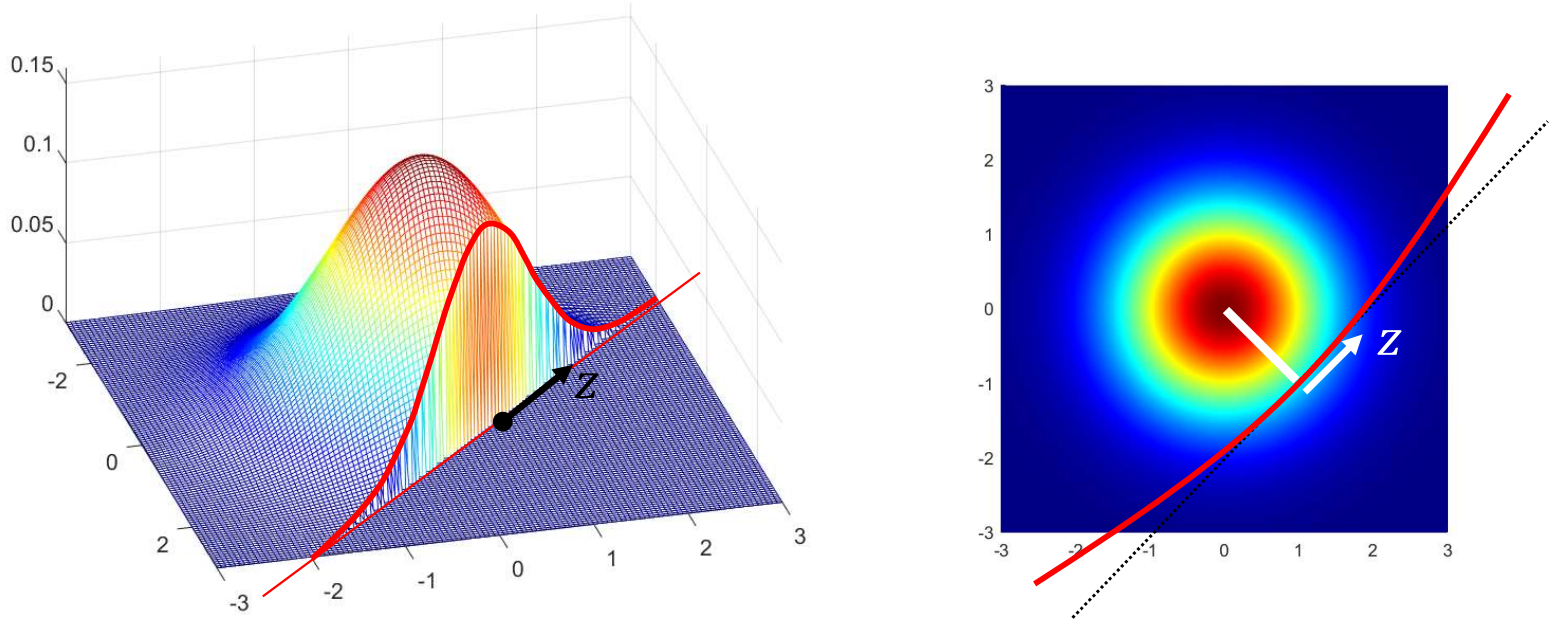
- Independence → $P(Z_1|Z_2) = P(Z_1)$
- $P(Z_1|Z_2) = N(0, I)$
- The conditional distribution of Z_1 given Z_2 is also an isotropic Gaussian regardless of the value of Z_2

A property of isotropic Gaussians



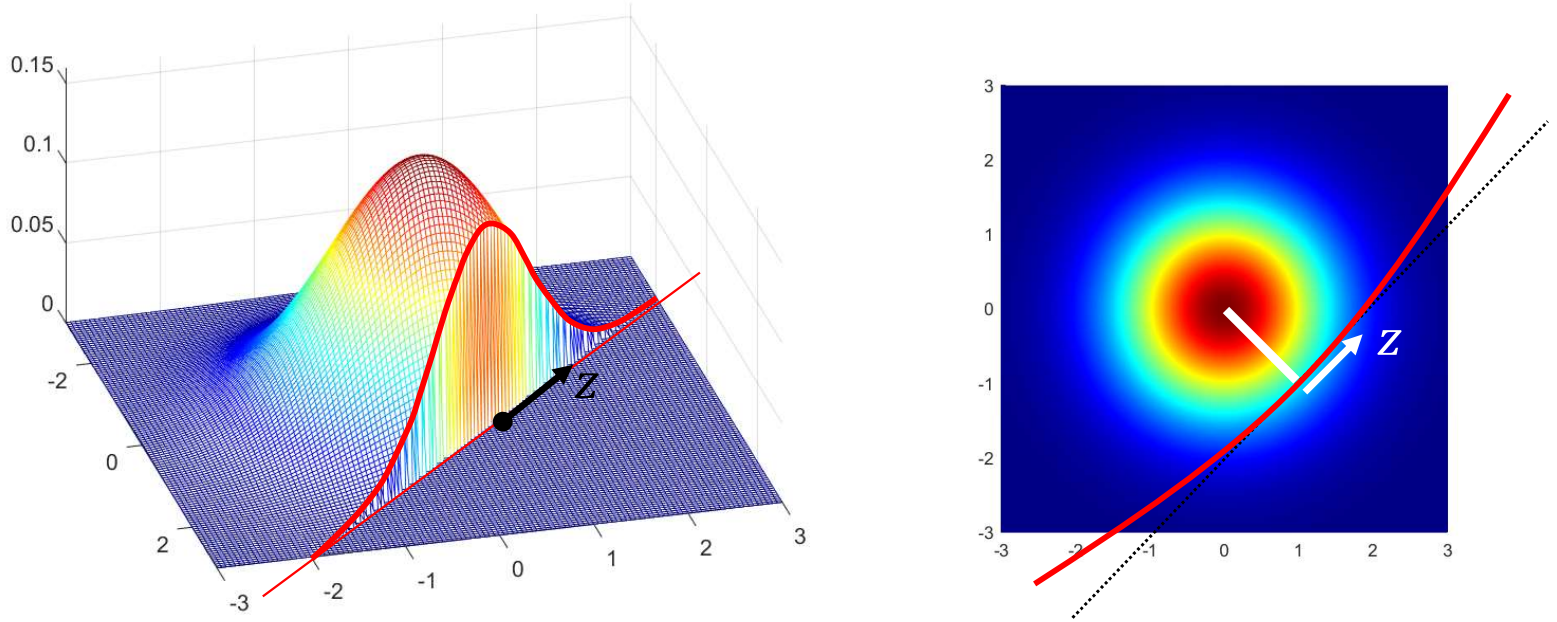
- This property will hold regardless of the line/hyperplane along which you consider the conditional
  - $P(Z|AZ = b) = N(0, I)$

A property of isotropic Gaussians



- This property will hold regardless of the line/hyperplane along which you consider the conditional
  - $P(Z|AZ = b) = N(0, I)$
- More generally, for "nearly linear" functions, the conditional distribution is still well approximated by a Gaussian (but the mean and variance may not be standard)
  - $P(Z|f(Z) = c) \approx N(\mu, \Sigma)$
  - "Nearly linear" : The curve $f(Z) = const$ does not deviate much from a hyperplane in high-probability regions of (Z)
    - $E\left(|f(Z) - linear(Z)|^2\right) < \epsilon$

A property of isotropic Gaussians



- More generally, for "nearly linear" functions
  - $P(Z|f(Z) = c) \approx N(\mu, \Sigma)$
- But μ and Σ will depend on c (and $f()$)

Poll 3 (@, @)

- The Z is a random vector with an isotropic Gaussian distribution, the conditional distribution of the projection of Z on any affine plane is also isotropic
  - True
  - False

- If Z is a random vector with an isotropic Gaussian distribution, the conditional distribution of Z on any curved surface is also approximately Gaussian for surfaces of low curvature
  - True
  - False

Poll 3

- The Z is a random vector with an isotropic Gaussian distribution, the conditional distribution of the projection of Z on any affine plane is also isotropic
  - **True**
  - False

- If Z is a random vector with an isotropic Gaussian distribution, the conditional distribution of Z on any curved surface is also approximately Gaussian for surfaces of low curvature
  - **True**
  - False

The variational autoencoder



- The *encoder* computes the *distribution* $Q(z|X)$ for any X
  - As an approximation to $P(z|X)$
- The *decoder* tries to convert a randomly sampled z from $Q(z|X)$ to X
  - Again, $Q(z|X)$ is intended to be a close approximation to $P(z|X)$

- **Training the encoder**:
  - Estimate θ to make the zs that can be encoded $\hat{X}$ values that are closer to X more probable

- **Training the decoder**:
  - Estimate ϕ to make the noise between $\hat{X} = D(z)$ and X more probable

- **Constraint on z:** Make the *average* $Q(z|X)$ as close to the standard Gaussian as possible

The variational autoencoder



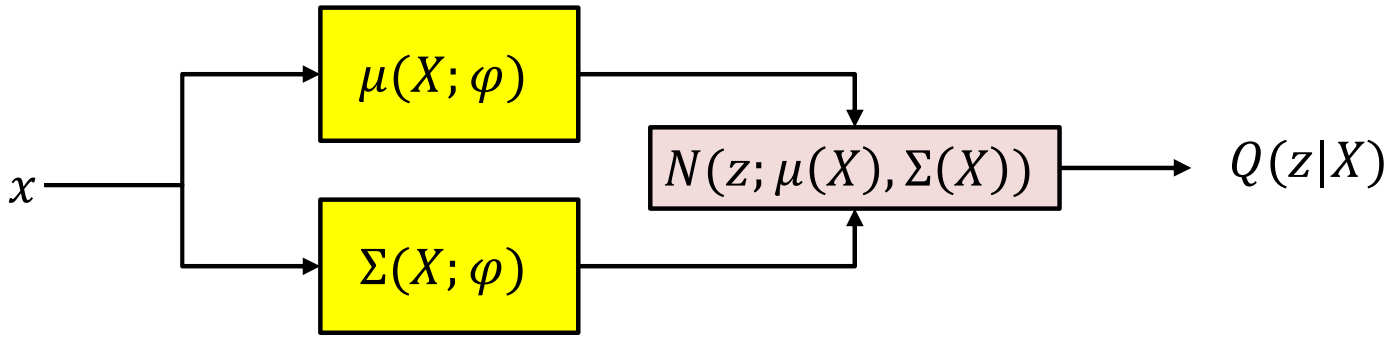
- The *encoder* computes the *distribution* $Q(z|X)$ for any X
  - As an approximation to $P(z|X)$
- The *decoder* tries to convert a randomly sampled z from $Q(z|X)$ to X
  - Again, $Q(z|X)$ is intended to be a close approximation to $P(z|X)$

- **Training the encoder**:
  - Estimate θ to make the zs that can be encoded $\hat{X}$ values that are closer to X more probable

- **Training the decoder**:
  - Estimate ϕ to make the noise between $\hat{X} = D(z)$ and X more probable

- **Constraint on z:** Make the *average* $Q(z|X)$ as close to the standard Gaussian as possible

Approximating $P(z|X)$



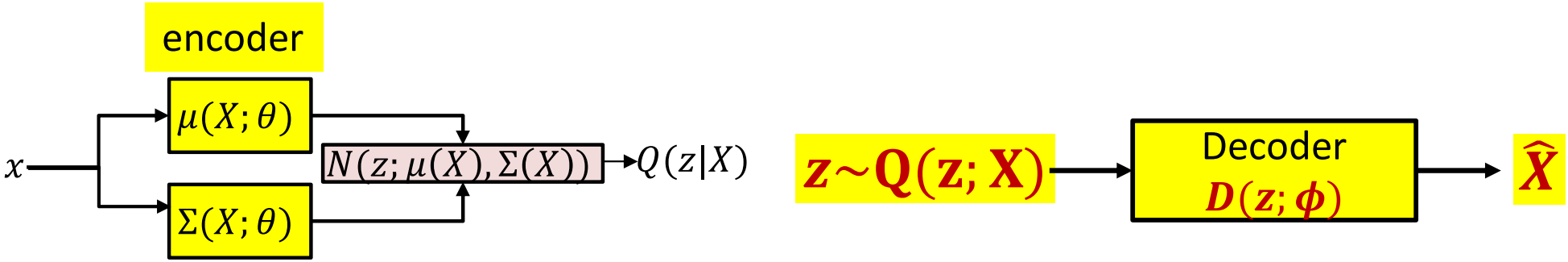
$\mu(x;\varphi)$ and $\Sigma(x;\varphi)$ are parametric functions of x, with parameters that we jointly represent as φ

- We approximate $P(z|X)$ as

$$P(z|X) \approx Q(z|X) = Gaussian\ N(z;\mu(X),\Sigma(X))$$

  - where $\mu(X;\varphi)$ and $\Sigma(X;\varphi)$ are estimated such that $Q(z|X)$ approximates $P(z|x)$ as closely as possible
  - For convenience, we will assume $\Sigma(X;\varphi)$ is a diagonal matrix, represented entirely by its diagonal elements
- We will use $Q(z|X)$ as our proxy for $P(z|X)$

The variational autoencoder



- The *encoder* computes the *distribution* $Q(z|X)$ for any X
- The *decoder* tries to convert a randomly sampled z from $Q(z|X)$ to X

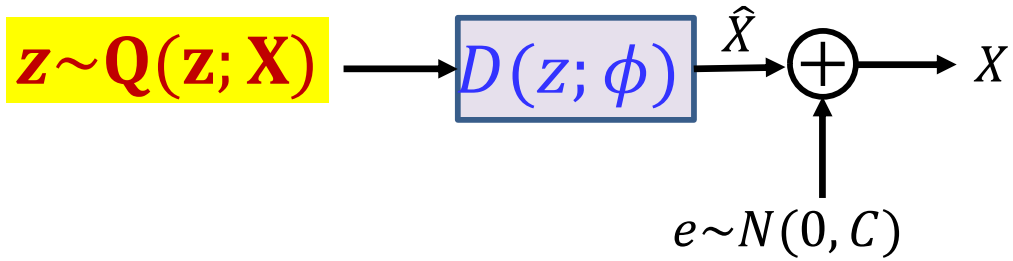
- **Training the encoder**:
  - Estimate $\boldsymbol{\theta}$ to make the zs that can be encoded to X more probable

- **Training the decoder**:
  - Estimate $\boldsymbol{\phi}$ to make the noise between $\hat{X} = D(z)$ and X more probable

- Constraint on z: Make the average $Q(z|X)$ as close to the standard Gaussian as possible

Training the encoder

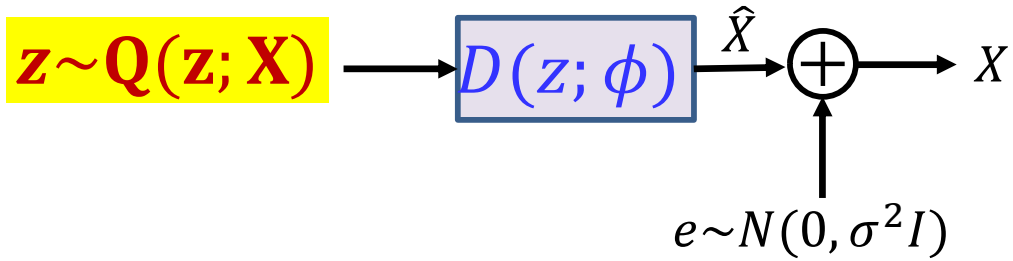
$$X = D(z;\phi) + e$$



$$\mathrm{P}(X|z) = N(X; D(z;\phi), C)$$

Training the encoder

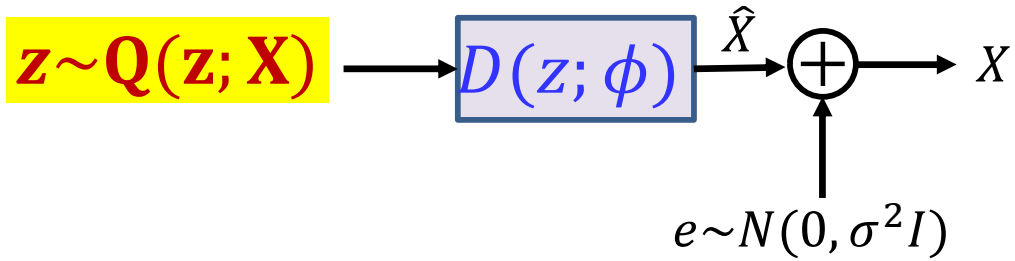
$$X = D(z;\phi) + e$$



$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

Training the encoder

$$X = D(z;\phi) + e$$

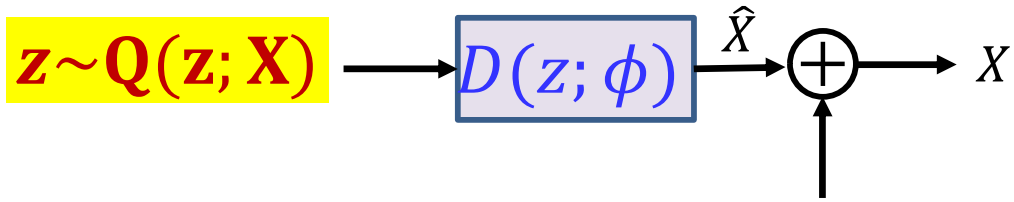


$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

$$-\frac{d}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\|X - D(z;\phi)\|^2$$

Training the encoder

$$X = D(z;\phi) + e$$

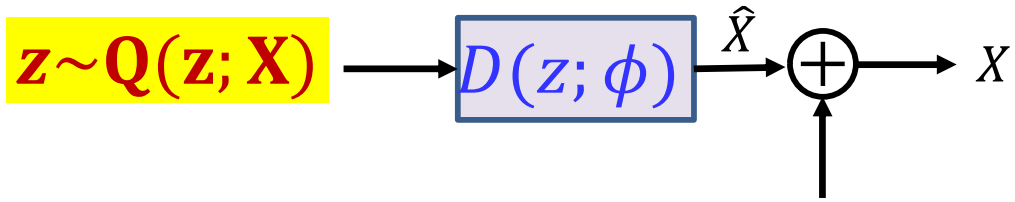


$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \sum_{(X,z)} -\frac{d}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\|X - D(z;\phi)\|^2$$

Training the encoder

$$X = D(z;\phi) + e$$



$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

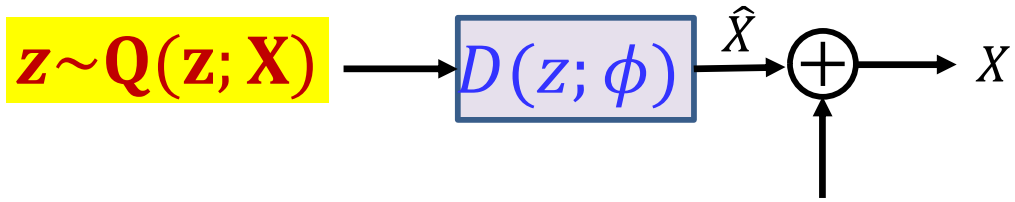
$$\theta^* = \underset{\theta}{\operatorname{argmax}} \sum_{(X,z)} -\frac{d}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\|X - D(z;\phi)\|^2$$

- We can learn the parameters using backpropagation, which minimizes the following loss

$$L(\theta, \phi, \sigma^2) = \sum_{(X,z)} d\log(\sigma^2) + \frac{1}{\sigma^2}\|X - D(z;\phi)\|^2$$

Training the encoder

$$X = D(z;\phi) + e$$



$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \sum_{(X,z)} -\frac{d}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\|X - D(z;\phi)\|^2$$

- We can learn the parameters using backpropagation, which minimizes the following loss

$$L(\theta, \phi, \sigma^2) = \sum_{(X,z)} d\log(\sigma^2) + \frac{1}{\sigma^2}\|X - D(z;\phi)\|^2$$

- Must minimize this with respect to θ

$$\theta^* = \underset{\theta}{\operatorname{argmin}}\, L(\theta, \phi, \sigma^2)$$

Sampling z

- *For each training input X, z is obtained as a sample from $N(z; \mu(X;\theta), \Sigma(X;\theta))$*

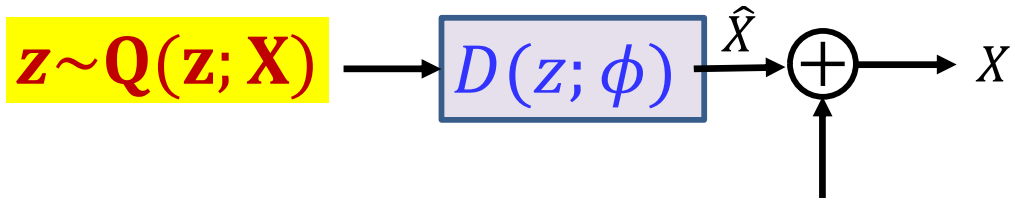
- We use a standard "reparametrization" step to sample z
  - Draw K-dimensional vector ε from $N(0, I)$
  - Compute $z = \mu(X;\theta) + \Sigma(X;\theta)^{0.5}\varepsilon$

Remember this one $\quad \nabla_\theta z = \nabla_\theta \mu(X;\theta) + diag(\varepsilon)\nabla_\theta \Sigma(X;\theta)^{0.5}$

This will be specific to X and to the *specific sample of* z for that X (via ε)

Training the encoder

$$X = D(z;\phi) + e$$



$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

$$L(\theta, \phi, \sigma^2) = d \log \sigma^2 + \sum_{(x,z)} \frac{1}{\sigma^2} \|X - D(z;\phi)\|^2$$

- $z = \mu(X;\theta) + \Sigma(X;\theta)^{0.5}\varepsilon$
- ε is sampled from $N(0, I)$

$$\theta^* = \underset{\theta}{\operatorname{argmin}}\, L(\theta, \phi, \sigma^2)$$

- The derivative of $L(\theta, \phi, \sigma^2)$ with respect to θ can be computed using the chain rule $\nabla_\theta L(\theta, \phi, \sigma^2) = \sum_{(X,z)} \nabla_z L(\theta, \phi, \sigma^2)\, \nabla_\theta z$
  - For use in backpropagation

The variational autoencoder



- The *encoder* computes the *distribution* $Q(z|X)$ for any X
- The *decoder* tries to convert a randomly sampled z from $Q(z|X)$ to X

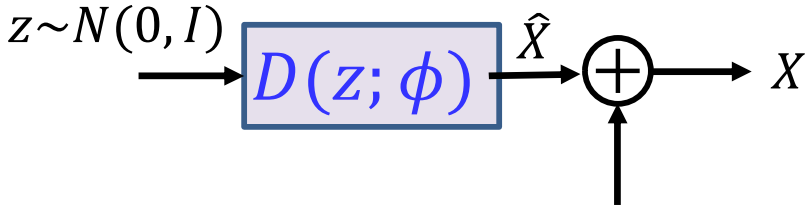
- **Training the encoder**:
  - Estimate $\boldsymbol{\theta}$ to make the zs that can be encoded to X more probable

- **Training the decoder**:
  - Estimate $\boldsymbol{\phi}$ to make the noise between $\hat{X} = D(z)$ and X more probable

- Constraint on z: Make the average $Q(z|X)$ as close to the standard Gaussian as possible

Training the decoder

$$X = D(z;\phi) + e$$



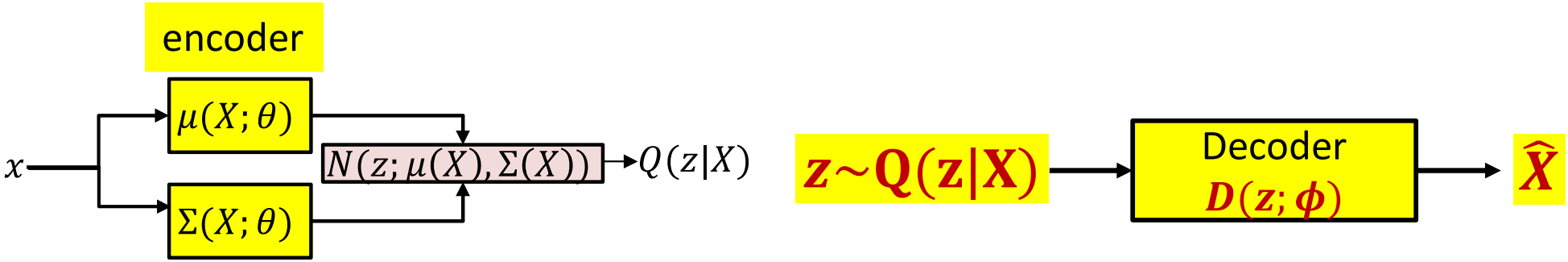
$$\mathrm{P}(X|z) = N(X; D(z;\phi), \sigma^2 I)$$

$$L(\theta, \phi, \sigma^2) = d \log \sigma^2 + \sum_{(X,z)} \frac{1}{\sigma^2} \|X - D(z;\phi)\|^2$$

$$\phi^*, {\sigma^2}^* = \underset{\phi,\sigma^2}{\operatorname{argmin}}\, L(\theta, \phi, \sigma^2)$$

- The derivative of this w.r.t $\phi$and σ^2 is trivially computed for backprop

The variational autoencoder



- The *encoder* computes the *distribution* $Q(z|X)$ for any X
- The *decoder* tries to convert a randomly sampled z from $Q(z|X)$ to X

- **Training the encoder**:
  - Estimate theta to make the zs that can be encoded to X more probable

- **Training the decoder**:
  - Estimate $\boldsymbol{\phi}$ to make the noise between $\hat{X} = D(z)$ and X more probable

- Constraint on z: Make the average $Q(z|X)$ as close to the standard Gaussian as possible

The constraint on P(z)

- The KL between $Q(z|X) = N(z; \mu(X;\theta), \Sigma(X;\theta))$ and the standard Gaussian $N(z; 0, I)$ works out to

$$KL(Q(z,x;\theta), N(z;0,I)) = \frac{1}{2}\left(tr\left(\Sigma(x;\theta)\right) + \mu(x;\theta)^T\mu(x;\theta) - d - \log|\Sigma(x;\theta)|\right)$$

  - This is a function of encoder parameters θ

The constraint on P(z)

- The KL between $Q(z|X) = N(z; \mu(X;\theta), \Sigma(X;\theta))$ and the standard Gaussian $N(z; 0, I)$ works out to

$$KL(Q(z|X;\theta), N(z;0,I)) = \frac{1}{2}\left(tr\left(\Sigma(x;\theta)\right) + \mu(X;\theta)^T\mu(X;\theta) - d - \log|\Sigma(X;\theta)|\right)$$

  - This is a function of encoder parameters θ
- The overall loss thus becomes:

$$L_{VAE}(\theta, \varphi, \sigma^2)$$

$$= \sum_X \left(tr\left(\Sigma(X;\theta)\right) + \mu(X;\theta)^T\mu(X;\theta) - d - \log|\Sigma(X;\theta)|\right) + \frac{1}{\sigma^2}\sum_{(x,z)\in[X,z]} \|(X - D(z;\phi))\|^2 + d\log\sigma^2$$

- This must be minimized to train the VAE
  - The derivatives of all terms w.r.t. are easily computed using backprop

The complete training pipeline

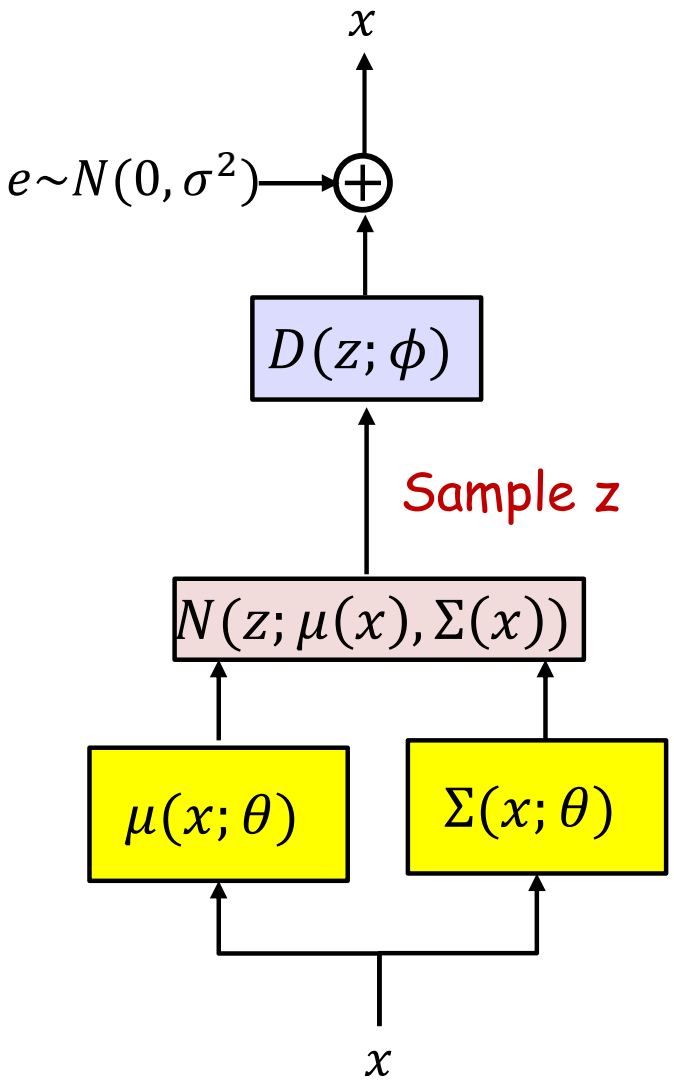
- Initialize θ and φ
- Iterate:
  - Sample $z_{x,\varepsilon}$ from $N(z;\mu(x;\varphi),\Sigma(x;\varphi))$ for each training instance
  - Reestimate $\theta, \varphi, \sigma^2$ from

$$L_{VAE}(\theta,\phi,\sigma^2)$$

$$= \sum_{x\in X}\Big(tr\big(\Sigma(x;\theta)\big) + \mu(x;\theta)^T\mu(x;\theta) - d$$

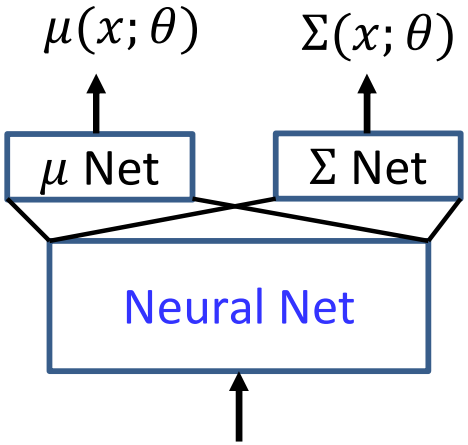
$$- \log|\Sigma(x;\theta)|\Big) + \frac{1}{\sigma^2}\sum_{(x,z)\in[X,Z]}\|(x - D(z;\phi))\|^2$$

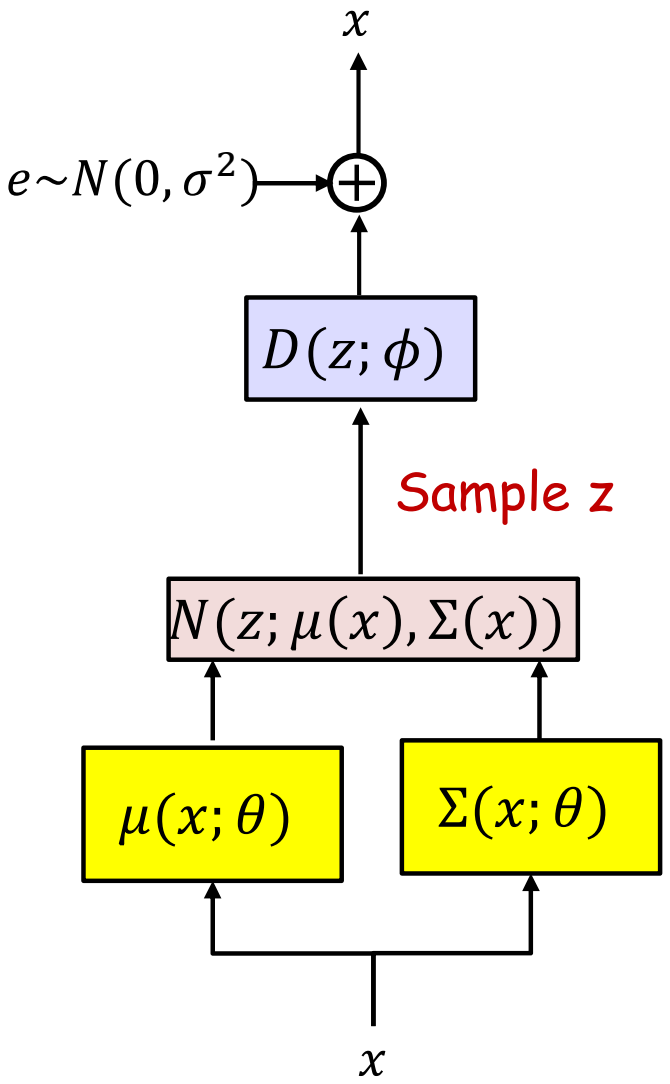
$$+ d\log\sigma^2$$



But where are the neural nets?

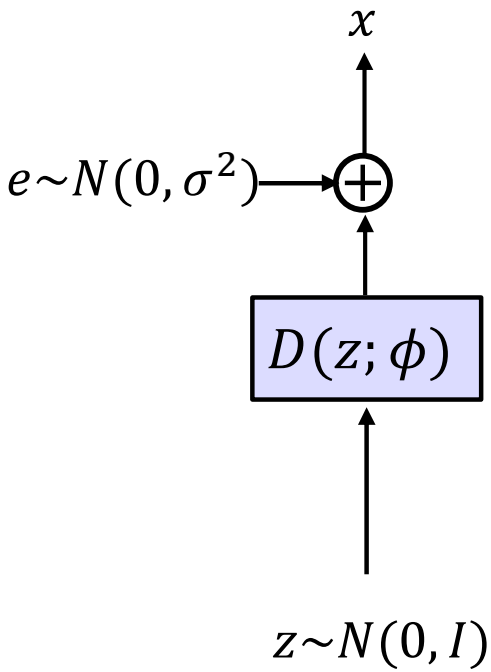
- $D(z;\theta)$ is a neural network
- $\mu(x;\theta)$ and $\Sigma(x;\theta)$ are generally modelled by a *common* network with two outputs
  - The combined parameters of the network are θ





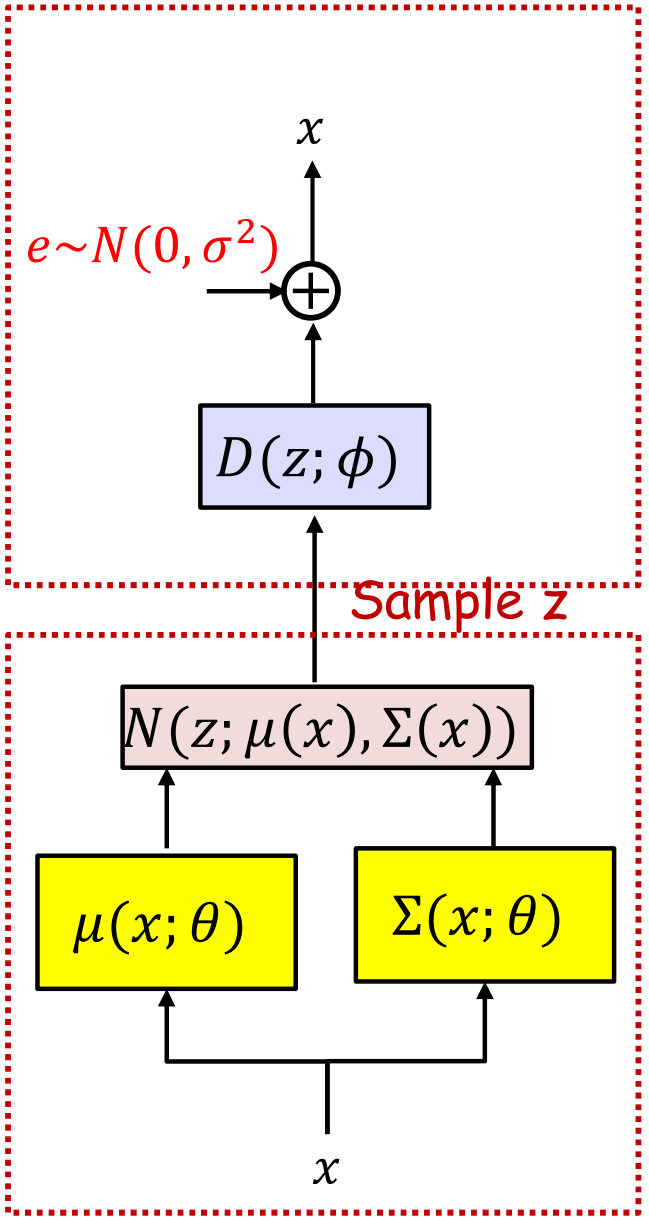
The VAE for generation

- Once trained the encoder can be discarded
- The rest of the network gives us a *generative* model for x
- Generating data using this part of the model should (ideally) give us data similar to the training data

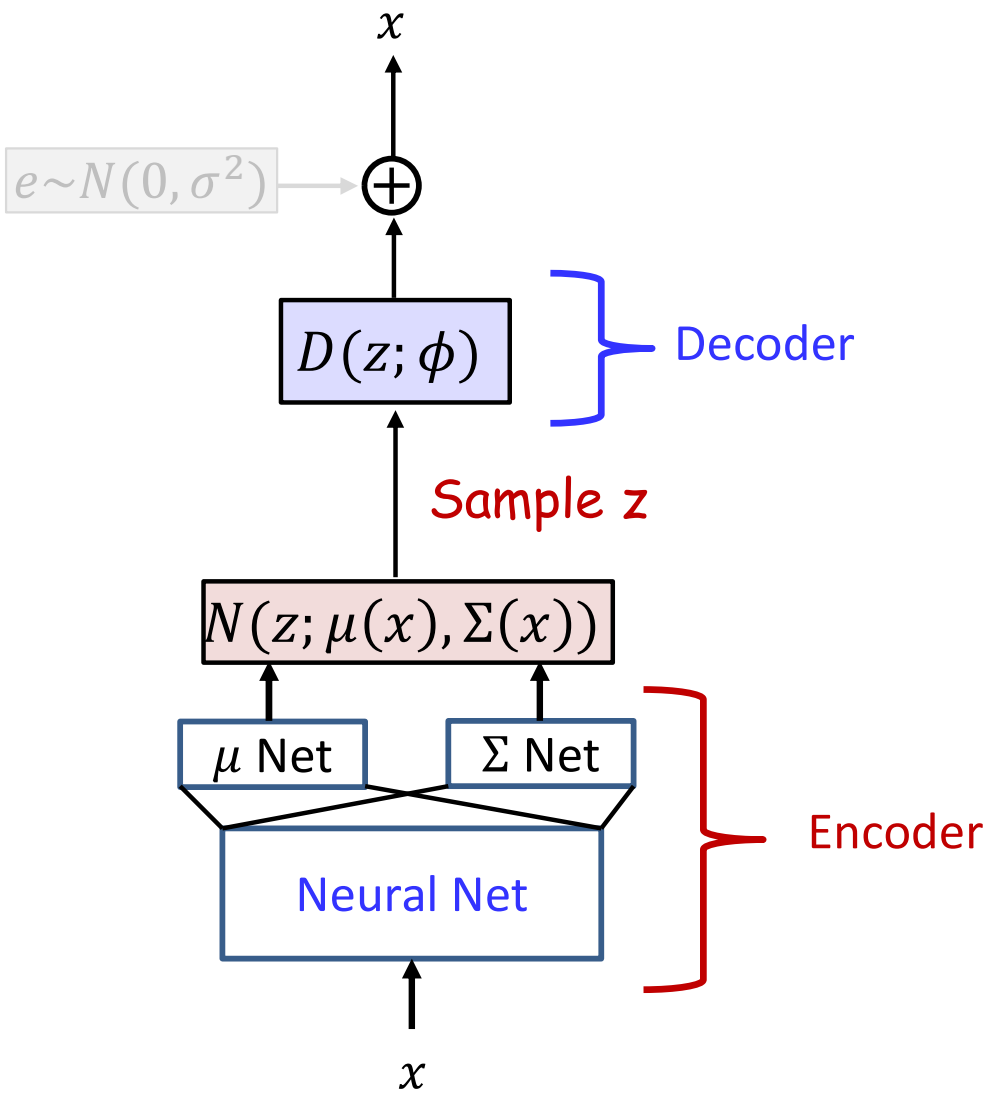


Recap: The VAE

- An autoencoder with statistical constraints on the hidden representation
  - The encoder is a statistical model that computes the parameters of a Gaussian
  - The decoder converts samples from the Gaussian back to the input
- The *decoder* is a generative model that, when excited by standard Gaussian inputs, generates samples similar to the training data



The Variational AutoEncoder



The decoder is the actual generative model.

The encoder is primarily needed for training.
It can also be used to generate the (approximate) distribution of latent space representations conditioned on specific inputs (much like a regular autoencoder).

z is a *latent-space* representation of the data.

$\mu(x)$ can also be used as an *expected latent* representation of x.

Poll 4 (@, @)

- A variational autoencoder is a generative extension of autoencoders that models probability distributions
  - True
  - False

- Select the true statements
  - The standard VAE assumes a latent representation that has an isotropic Gaussian PDF
  - The VAE model requires addition of Gaussian noise to the output of a regularized AE in order to permit the output to fill space beyond a lower-dimensional manifold
  - The decoder of the VAE can be used to generate samples from the distribution of the data it is trained on, if the input to the decoder is drawn from a standard Gaussian

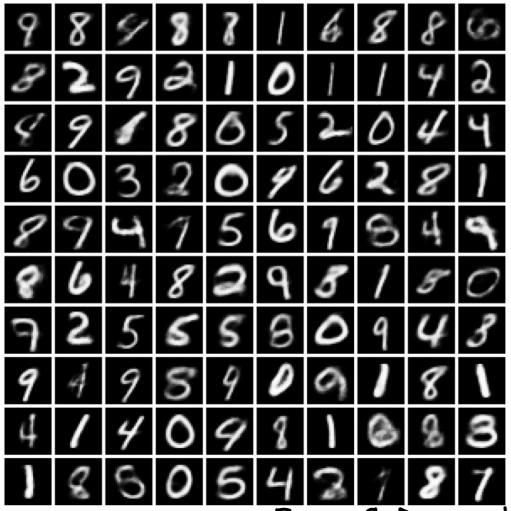
Poll 4

- A variational autoencoder is a generative extension of autoencoders that models probability distributions
  - **True**
  - False

- Select the true statements
  - **The standard VAE assumes a latent representation that has an isotropic Gaussian PDF**
  - **The VAE model requires addition of Gaussian noise to the output of a regularized AE in order to permit the output to fill space beyond a lower-dimensional manifold**
  - **The decoder of the VAE can be used to generate samples from the distribution of the data it is trained on, if the input to the decoder is drawn from a standard Gaussian**

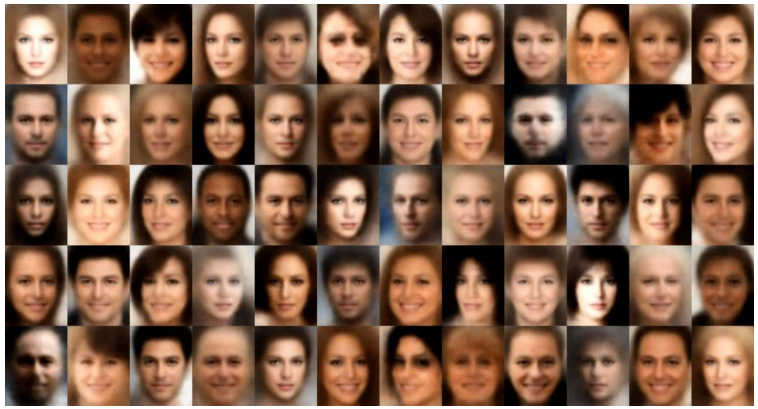
VAE examples

- Top: VAE trained on MNIST and used to generate new data



From C. Doersch

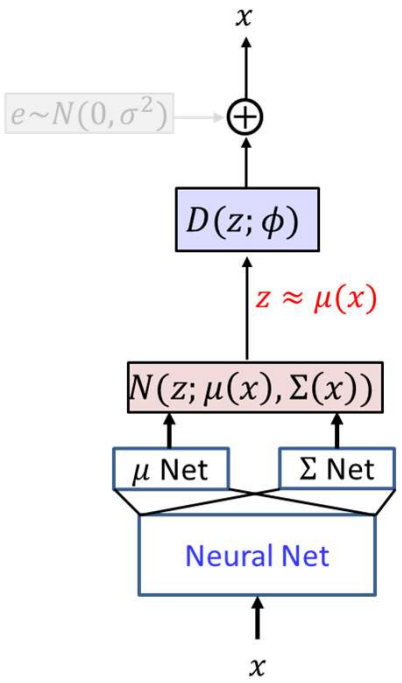
- Below: VAE trained on faces, and used to generate new data



From J. Rocco

VAE and latent spaces

- The latent space z often captures underlying structure in the data x in a smooth manner
  - Varying z continuously in different directions can result in plausible variations in the drawn output



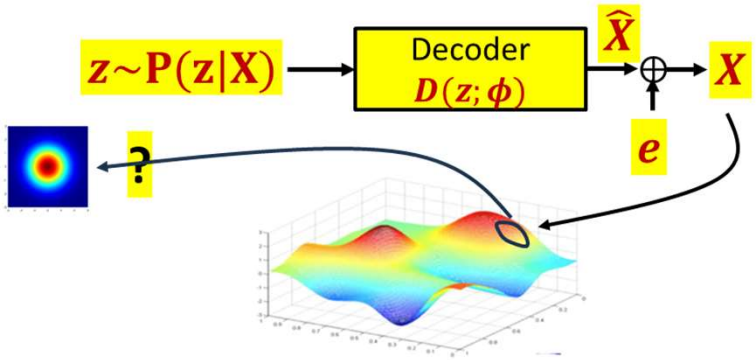
- Reproductions of an input x can be manipulated by wiggling z around its expected value $\mu(x)$

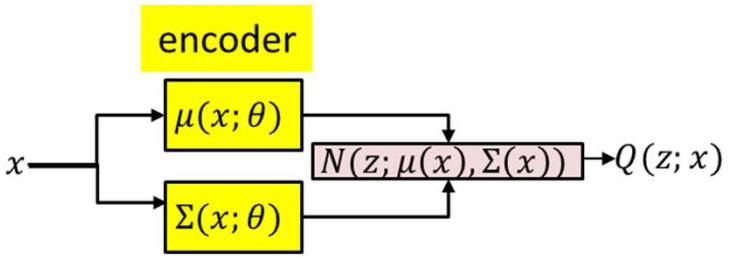


VAE conclusions

- Simple statistical extension of the autoencoder
- Excellent generative models for the distribution of data $P(x)$
  - Various extensions such as Conditional VAEs, which model *conditional* distributions, such as $P(x|y)$
    - Straight-forward extension where the conditioning variable y is an additional input to the encoder and decoder
- Read the literature on the topic, it is vast

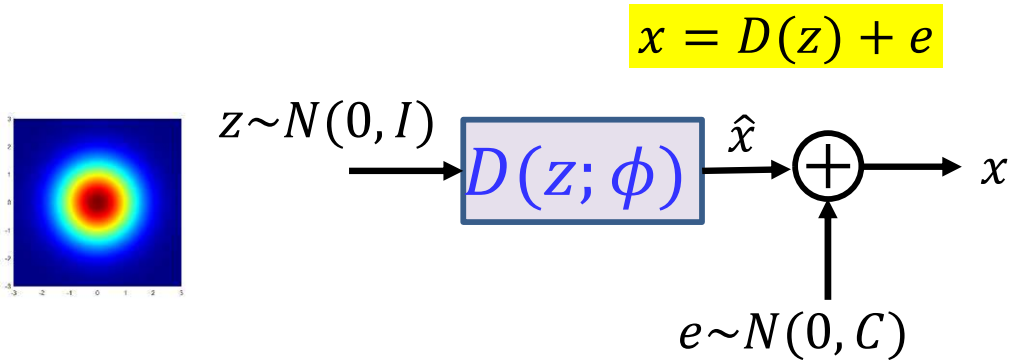
VAE limitations





- The decoder is not generally invertible
  - Cannot get a closed form for $P(\boldsymbol{z}|\boldsymbol{X})$
- This requires an Encoder that *approximates* $P(\boldsymbol{z}|\boldsymbol{X})$
- Solution: "**Normalizing flow models**" – Invertible decoders that give you closed forms for $P(\boldsymbol{z}|\boldsymbol{X})$

VAE limitations





- The decoder must transform a standard Gaussian all the way to the target distribution in one step
  - This is often too large a gap
  - Results in blurry, unsatisfactory generation

- Solution: Make the seed model closer to the final distribution
  - Make it itself a VAE
    - Whose seed model is a VAE
      - Whose seed model is a VAE
        - » …
  - **AKA Diffusion models**

Next Class!

